BOOK

SOFTWARE ENGINEERING

Mr. Hussein Younis Computer Systems Engineering Department

Last updated 11 November 2026

Chapter 1 Introduction to Software Engineering

1 Defining the Discipline

To understand **software engineering**, we must first dissect its two constituent components: *software* and *engineering*.

- **Software:** Far beyond simple, standalone executable code, software is comprehensively defined as:

$$\text{Software} = \text{Instructions} + \text{Data Structures} + \text{Documents}$$

  It comprises executable code that serves a computational purpose, the structural data architectures it manipulates, and the technical documentation required to operate, maintain, and understand it. When built to fulfill a specific operational requirement, it constitutes a **software product**.

- **Engineering:** The branch of science and technology concerned with the systematic design, building, and utilization of engines, machines, and structured solutions. It applies scientific principles, tools, and formal methods to solve both simple and complex problems efficiently and cost-effectively.

According to the IEEE Computer Society's Guide to the Software Engineering Body of Knowledge (SWEBOK), software engineering is formally defined as:

> 📖 DEFINITION 1: **Software Engineering(IEEE)**
>
> The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software.

> 📖 DEFINITION 2: **Software Engineering**
>
> Software engineering is a discipline within computer science focused on applying **systematic, disciplined, and quantifiable engineering principles** to the development, operation, maintenance, and evolution of software systems.

> ⓘ **Note**
>
> **The Indispensability of Principles:** While small programs can be developed casually without software engineering principles and methodologies, structured engineering approaches are **indispensable** if we want to achieve high-quality software in a reliable, predictable, and cost-effective manner.

1.1 Lehman's Classification of Software Types

Not all software behaves or evolves in the same way. Computer scientist Manny Lehman classified software into three distinct categories based on how they interact with their environment and how their requirements evolve over time:

- **S-Type (Static / Specified):**
  - **Characteristics:** Built with a well-defined specification and an obvious, unchanging solution. Once written, the program rarely needs fundamental alterations.
  - **Example:** A standard calculator program designed to perform basic arithmetic (addition, subtraction, multiplication, division). Once the logic for these operations is correctly implemented, the problem is permanently solved.
- **P-Type (Procedural):**
  - **Characteristics:** The controlling procedures and rules are well-defined, but the exact solution depends entirely on dynamic runtime inputs and opponent actions.
  - **Example:** Chess-playing software. While the rules of chess are completely fixed, the computer's next move cannot be predetermined statically; it must be calculated dynamically based on the opponent's moves during actual gameplay.
- **E-Type (Evolving):**
  - **Characteristics:** Represents the vast majority of real-world commercial software. Both the problem specification and the solution are tightly coupled to a constantly changing operational and social environment.
  - **Example:** E-commerce systems like Amazon. These platforms continuously evolve hour by hour to adapt to shifting economic environments, updated user requirements, changing business rules, and new feature requests.

2 Complexity Management: Abstraction and Decomposition

Without software engineering principles, developing large-scale commercial programs becomes extraordinarily difficult because the complexity and difficulty of programs grow rapidly as they get bigger. To control this growth in complexity, software engineering relies on two main techniques:

- **Abstraction:** Simplifying a complex problem by ignoring minor details and focusing only on the main concepts needed for a specific task. Once the simpler big-picture problem is solved, the omitted details can be added back in at lower levels of design.

- **Decomposition:** Breaking a large, complicated problem down into several smaller, manageable parts that can be solved one by one. The trick is to split the problem so that each smaller piece can be worked on independently, and their solutions fit together seamlessly. A good design keeps the connections between these pieces as simple and minimal as possible so they don't depend too heavily on each other.

> 🎬 SCENARIO 1: **Real-World Example: Building a University Student Portal**
>
> To see how these principles work in practice, consider building a university registration and student portal:
>
> - **Applying Abstraction:** When designing the tuition fee payment screen, developers use abstraction to hide the messy, low-level implementation details behind a clean, unified boundary. Instead of dealing with the fact that each local bank has its own distinct internal workings—such as whether a bank uses a SOAP protocol, a REST API with specific JSON payloads, or a completely different messaging format—the core registration system doesn't need to worry about any of that behind-the-scenes complexity. It simply sends a standardized request, like asking to process a payment, and receives back a straightforward status response indicating success or failure. This keeps the core application logic simple, independent, and much easier to maintain.
> - **Applying Decomposition:** Instead of writing the entire portal as one massive, tangled block of code, engineers break it down into independent modules:
>
>   1. **Student Login & Authentication Module** (Handles passwords and user security).
>   2. **Course Prerequisite Checker** (Checks if a student has passed required classes before letting them register).
>   3. **Financial Clearance Ledger** (Tracks tuition balances and holds).
>   4. **Class Schedule Manager** (Manages room capacities and time slots).
>
> By separating the system into these independent modules, developers can update the payment system or fix a scheduling bug in one module without accidentally breaking the rest of the portal.

2.1 Architectural Breakdown: High-Level vs. Low-Level Design

When transforming software requirements into a robust blueprint during the design phase, software engineering divides the architecture into two distinct tiers of granularity: High-Level Design (HLD) and Low-Level Design (LLD). This separation ensures that system-wide structural decisions are resolved before committing to module-specific implementations and team resource allocations.

- **High-Level Design (HLD):** Often referred to as macro-level or architectural design, HLD focuses on the overarching structure of the system rather than internal minutiae.
    - **Platform Determination:** Decides the target environments where the system will operate (e.g., choosing whether the application will deploy as a web platform, a native mobile app for Android and iOS, or a hybrid solution).
    - **Module Isolation:** Divides the massive system into major functional components (modules) and maps out how they interact and exchange data with one another.
    - **Team Independence:** At the high-level stage, architectural boundaries are drawn without yet assigning individual team members or specific developers to those modules.
- **Low-Level Design (LLD):** Often referred to as micro-level or detailed design, LLD drills down into the internal mechanics of each individual module defined during HLD.
    - **Granular Component Design:** Specifies internal logic, data structures, algorithm workflows, and detailed interface signatures for every module.
    - **Resource Assignment:** Allocates specific roles and responsibilities to individual team members and developers within those modules (e.g., assigning a particular developer as a module lead or component designer).

> ⓘ **Note**
>
> High-level design shapes the big-picture environment and communication pathways of the software, while low-level design bridges the architecture directly into executable tasks and team assignments.

3 The Need for Software Engineering

While a small program or script can be successfully written casually by an individual without formal methods, developing large-scale commercial or industrial software requires a disciplined engineering framework. The need for software engineering arises primarily because of the ever-changing nature of user requirements and the dynamic environments in which software operates.

The core drivers that make software engineering indispensable include:

- **Managing Large-Scale Systems:** Just as building a massive bridge or skyscraper requires rigorous structural engineering rather than the intuitive methods used to build a small wall, large-scale software systems—comprising multiple interconnected modules and thousands of concurrent transactions—require systematic processes to manage their immense structural complexity.

- **Ensuring Scalability:** If software development is not rooted in scientific and engineering principles, expanding or scaling an existing system becomes so difficult and brittle that it is often faster and cheaper to rewrite the software from scratch rather than modify it.
- **Controlling Costs:** While the hardware industry has successfully driven down the cost of physical computers and electronic components through advanced manufacturing, software development costs remain high—and often spiral out of control—unless a disciplined, cost-effective engineering process is adopted.
- **Adapting to Dynamic Environments:** Software does not operate in a vacuum; it constantly adapts to changing user needs, updated operating systems, and evolving business rules. Because software must frequently evolve, structured engineering practices are essential to handle ongoing enhancements safely.
- **Driving Quality Management:** Adopting a repeatable, standardized development process ensures that the resulting software product meets rigorous quality benchmarks, remaining functional, reliable, secure, and maintainable over its entire lifecycle.

4 Foundations of Software Engineering

4.1 The Historical Context: The "Software Crisis"

Historically, software engineering as a distinct field emerged in the late 1960s. During the first generation of computing, software took a back seat; the primary focus was on building hardware capable of solving a limited set of scientific and military problems. However, as hardware advancements made computers smaller, faster, and more accessible to universities and businesses, users began demanding complex, diverse commercial applications (such as payroll, accounting, and inventory management).

Human programming techniques, which treated software development as an ad-hoc art or individual craft, could not keep up with this growing demand. The industry reached a critical bottleneck known as the **Software Crisis**.

4.1.1 The 1968 NATO Conference

The crisis was officially brought to light in October 1968, when around 50 eminent computer scientists gathered in Garmisch, Germany, for a NATO-sponsored conference. The purpose of the meeting was to address the severe shortcomings in software production. The conference produced a landmark 130-page report advocating for software development based on practical and theoretical principles, drawing inspiration from traditional engineering disciplines like civil and mechanical engineering.

While the phrase "Software Engineering" had been used informally a few times prior (notably by Margaret Hamilton during the Apollo program), the NATO Science Committee deliberately chose it as

the title of the conference. This sparked a provocative and necessary debate: *Could the chaotic process of writing software be disciplined, structured, and measured like building bridges or airplanes?*

The conference officially diagnosed the "Software Crisis," characterized by systems that were:

- Consistently delivered over budget and severely behind schedule.
- Highly unreliable, buggy, and nearly impossible to maintain.
- So complex that no single programmer could fully understand them.

Crucially, the 1968 conference shifted the industry's focus from merely "writing code" to managing the entire **Software Lifecycle**. It introduced foundational concepts that remain vital today:

- **Software Design:** Planning the architecture before writing code.
- **Software Production:** Managing teams, deadlines, and tracking progress.
- **Software Maintenance:** Recognizing that fixing and updating code post-delivery is a massive, long-term endeavor.



Figure 1: Scientists at the 1968 NATO conference on software engineering. Reproduction kindly authorized by Prof. Robert McClure.

4.2 The Unique Nature of Software: "No Silver Bullet"

More than half a century after the NATO Conference, techniques for software construction have improved remarkably. We now utilize design patterns, reusable libraries, automated testing, and robust frameworks to avoid reinventing the wheel. However, software development possesses unique characteristics that separate it from traditional hardware engineering.

Frederick Brooks, a pioneer in Software Engineering and recipient of the 1999 Turing Award, underscored this fact in his seminal 1987 essay, "*No Silver Bullet: Essence and Accident in Software Engineering.*" Brooks argued that there is no single magical technology or management technique (a "silver bullet") that will miraculously improve software productivity by an order of magnitude.

Brooks categorized the difficulties of software engineering into two types: Essential Difficulties and Accidental Difficulties.

> ⓘ **Note**
>
> the term silver bullet, as used in Brooks's essay title, refers to a magical solution to a complex problem. Brooks argued that due to the essential difficulties, we should not anticipate the existence of silver bullets in the field. Nonetheless, novel software technologies are often marketed as if they were, in fact, silver bullets.

4.2.1 Essential Difficulties

These are intrinsic to the very nature of software and are unlikely to be solved by novel technologies. Brooks identified four primary essential difficulties:

1. **Complexity:** Software is one of the most intricate human-made constructions. Unlike physical objects, software components scale non-linearly, making large systems inherently complex.
2. **Conformity:** Software must adapt continuously to its ever-changing environment (e.g., new operating systems, hardware, or tax legislation). Unlike the laws of physics, the "laws" of software's environment change due to human decisions.
3. **Changeability:** There is constant pressure for software to evolve. The more successful a software product is, the more likely it is to attract requests for new features and modifications.
4. **Invisibility:** Software is inherently abstract. It is challenging to visualize its geometric structure or spatial complexity, unlike a car or a building, which can be easily evaluated by consumers.

> **Note**
>
> *Challenges 2, 3, and 4 are largely exclusive to software. When environmental laws change, car manufacturers have years to comply, and a manufactured car does not spontaneously receive new functionalities.*

4.2.2 Accidental Difficulties

These are difficulties not inherent to the software itself, but rather attached to the current tools and practices used to produce it. These *can* be addressed through better training, tools, and technologies. Examples include:

- An IDE (Integrated Development Environment) that frequently crashes.
- A compiler with cryptic, unhelpful error messages.
- A third-party framework lacking proper documentation.
- A web application with a confusing, unintuitive user interface.

While eliminating accidental difficulties improves developer experience, Brooks warned that it will not solve the fundamental *essential* complexities of building large software systems.

4.2.3 The Scale and Complexity of Modern Systems

The essential difficulty of **complexity** highlighted by Brooks is best illustrated by the sheer scale of modern software systems. Today, the open-source projects supported by the Linux Foundation comprise over 1.7 billion lines of code and involve 65,000 active developers (Linux Foundation, 2023). Specifically, the Linux kernel—the core component of the operating system—contains more than 30 million lines of code. Similarly, tech giants like Google have reported that their internal systems consist of billions of lines of code.

Managing systems of this magnitude, ensuring they remain reliable, secure, and adaptable over decades, is the ultimate challenge of our field. It is precisely because of this immense scale and complexity that Software Engineering remains not just a technical skill, but a critical, disciplined science essential to the modern world.

5 The Economics of Software Development

5.1 The Cost of Change Curve (Boehm's Curve)

>

> The earlier you catch a defect in the software development lifecycle, the cheaper it is to fix it.
>
> — Barry Boehm

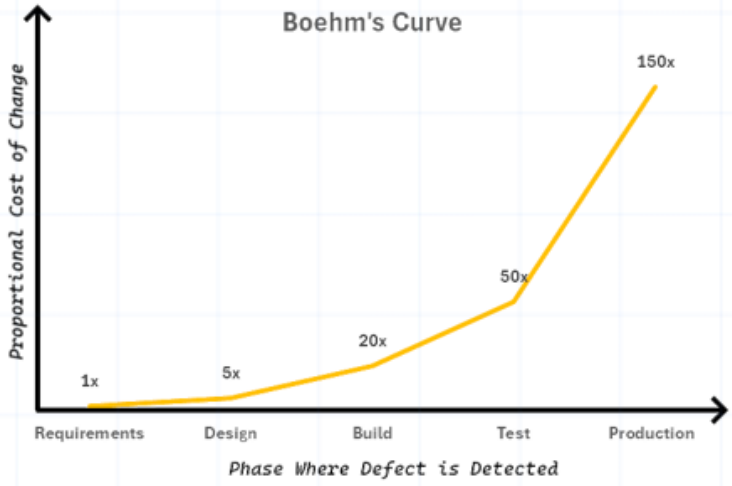


Figure 2: Boehm's Curve

Barry Boehm's theorem mathematically demonstrates why skipping disciplined engineering practices is financially ruinous across development phases:

- **Requirements Phase:** Finding and fixing an error during requirements costs a baseline of **1x**.
- **Design Phase:** Finding that same error during design costs roughly **5x**.
- **Build Phase:** Finding it during building and coding costs **20x**.
- **Test Phase:** Finding it during testing costs **50x**.
- **Production / Post-Release:** Fixing a structural error after release escalates dramatically to **150x** the original baseline.

Rigorous requirements gathering and architecture design catch critical flaws early when they are cheapest to mitigate.

5.2 Distribution of Software Lifecycle Costs

Coding and construction represent only a fraction of total expenditures. Following Ian Sommerville's framework, software costs are distributed as follows:

- **Development (Specification, Design, Coding, Testing):** Accounts for approximately **30–40%** of the total budget.
- **Evolution (Maintenance, Upgrades, Bug Fixes):** Accounts for **60–70%+** of the total lifecycle budget.

Without strict software engineering practices—such as documentation, clean architecture, and modularity—the evolution phase becomes prohibitively expensive, eventually draining organizational resources just to keep legacy systems operational.

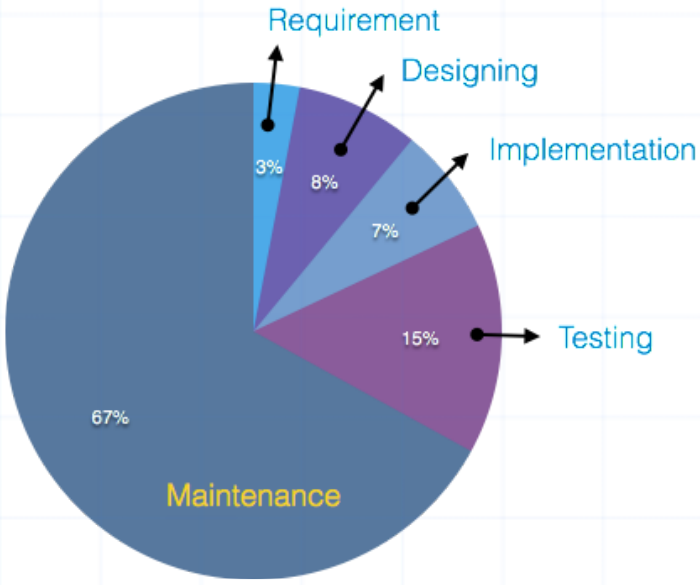


Figure 3: Costs of software development stages

6 Characteristics of Good Software

A high-quality software product can be judged by what it offers and how well it can be used. Well-engineered and crafted software must satisfy criteria across three vital dimensions: **Operational**, **Transitional**, and **Maintenance**.

6.1 Operational Qualities

This dimension measures how well the software performs during daily execution and active use. Key metrics include:

- **Correctness:** The extent to which the software satisfies its specified requirements and accurately executes its intended user tasks.
- **Efficiency:** How well the software utilizes system resources (such as CPU cycles, memory allocation, and network bandwidth) without unnecessary waste.
- **Dependability & Reliability:** The trustworthiness of the system, ensuring it functions without catastrophic failures over a given period.
- **Security & Safety:** Protection of data assets against unauthorized access, alongside ensuring the system does not cause physical, financial, or operational harm.
- **Usability & Budget:** The ease with which users can interact with the system, balanced against strict adherence to financial development constraints.

6.2 Transitional Qualities

This dimension evaluates how smoothly the software adapts when it is transitioned from one operating platform, environment, or organization to another:

- **Portability:** The ability of the software to run across different hardware platforms or operating systems with minimal modification.
- **Interoperability:** The capability of the system to communicate and exchange data seamlessly with other external software applications or third-party legacy services.
- **Reusability:** The extent to which parts of the software code and architecture can be packaged and reused in building other systems.
- **Adaptability:** The readiness of the system to adjust smoothly to emerging technological shifts.

6.3 Maintenance Qualities

This dimension addresses how easily a system can maintain itself and evolve within an ever-changing operational environment:

- **Modularity:** Decomposing the system into discrete, loosely-coupled components so that changes in one area do not ripple across others.
- **Flexibility & Maintainability:** The ease with which bugs can be corrected, features enhanced, or components modified over time.
- **Scalability:** The capability of the system to gracefully handle increasing loads, larger databases, or expanded user volumes without performance degradation.

7 Topics of Study

To present the topics of study in software engineering, we refer to the *Guide to the Software Engineering Body of Knowledge*, also known as the SWEBOK (link). This comprehensive report is organized by the IEEE Computer Society, an internationally recognized scientific society. It is crafted with the expertise of numerous researchers and industry professionals. The aim of the report is to document and compile the body of knowledge that defines the field we now recognize as software engineering.

The SWEBOK defines the following key areas of Software Engineering:

1. Software Requirements
2. Software Architecture
3. Software Design

4. Software Construction
5. Software Testing
6. Software Engineering Operations
7. Software Maintenance
8. Software Configuration Management
9. Software Engineering Management
10. Software Engineering Processes
11. Software Engineering Models and Methods
12. Software Quality
13. Software Security
14. Software Engineering Professional Practice
15. Software Engineering Economics

Chapter 2 Software Development Processes & Agile Methodologies

> *In software development, perfect is a verb, not an adjective. There is no perfect process. There is no perfect design. There are no perfect stories. You can, however, perfect your process, your design, and your stories.*
>
> — Kent Beck

1 Understanding Software Engineering Fundamentals

Building modern, high-quality software requires more than just writing code; it demands a structured approach to manage complexity, align with business goals, and coordinate team efforts. To understand how software development is organized, it helps to examine three foundational concepts in logical order: the **Software Development Life Cycle (SDLC)**, **Methods (or Methodologies)**, and **Processes**.

1.1 The Software Development Life Cycle (SDLC): The Macro-Level Roadmap

At the highest level, every software project is governed by the **Software Development Life Cycle (SDLC)**. The SDLC provides the macro-level conceptual framework that outlines the entire journey of a software product from conception to decommissioning. It defines the chronological stages a project must navigate—such as planning, analysis, design, development (implementation), testing, deployment, and ongoing maintenance. Think of the SDLC as the overarching architectural blueprint for the project.

> 📖 DEFINITION 1: **Software Development Life Cycle (SDLC)**
>
> The Software Development Life Cycle (SDLC) is a structured process used to plan, design, develop, test, deploy, and maintain software. It ensures a systematic workflow and helps align software development with business goals and user requirements.

1.1.1 Benefits of the software development life cycle

The Software Development Life Cycle (SDLC) provides a structured roadmap for software development by breaking the entire project journey down into distinct phases—such as planning, analysis, design, development, testing, deployment, and ongoing maintenance. Following a disciplined SDLC helps

organizations overcome traditional development challenges, mitigate technical risks, and secure several core benefits:

- **More visibility:** Keep stakeholders informed, aligned, and involved throughout the entire development process.

`Example: During the development of an e-commerce platform using an SDLC model, regular milestone reviews allow the marketing, sales, and logistics teams to log in and inspect the staging environment. Because progress is mapped to transparent phases, stakeholders are never surprised by last-minute delays and can coordinate inventory and advertising campaigns well in advance.`

- **Greater efficiency:** Demystify each step of software creation, making it easier for teams to meet project deadlines and system requirements.

`Example: When a financial institution builds a new mobile banking app, following a structured SDLC ensures that the development team does not waste time writing backend transaction code before the security architecture and database schema are approved. Every phase has clear entry and exit criteria, keeping the team moving forward smoothly from task to task.`

- **Higher consistency:** Ensure that every team member adheres to standardized processes, best practices, and organizational norms.

`Example: In a large software enterprise where dozens of developers work on the same codebase, enforcing an SDLC guarantees that every team member follows the exact same branching strategy, code review policy, and automated testing standard before code is merged into production.`

- **Extra time saved:** Avoid confusion and costly misunderstandings by defining clear requirements at the very start of the development process.

`Example: A healthcare software team spends adequate time during the initial requirements and analysis phase to document compliance rules for patient data handling. By nailing down these strict specifications early, they avoid spending months rewriting core components later due to regulatory misunderstandings.`

- **Lower costs:** Minimize wasted time and inflated budgets by reducing the likelihood of unexpected software errors and extensive reworks.

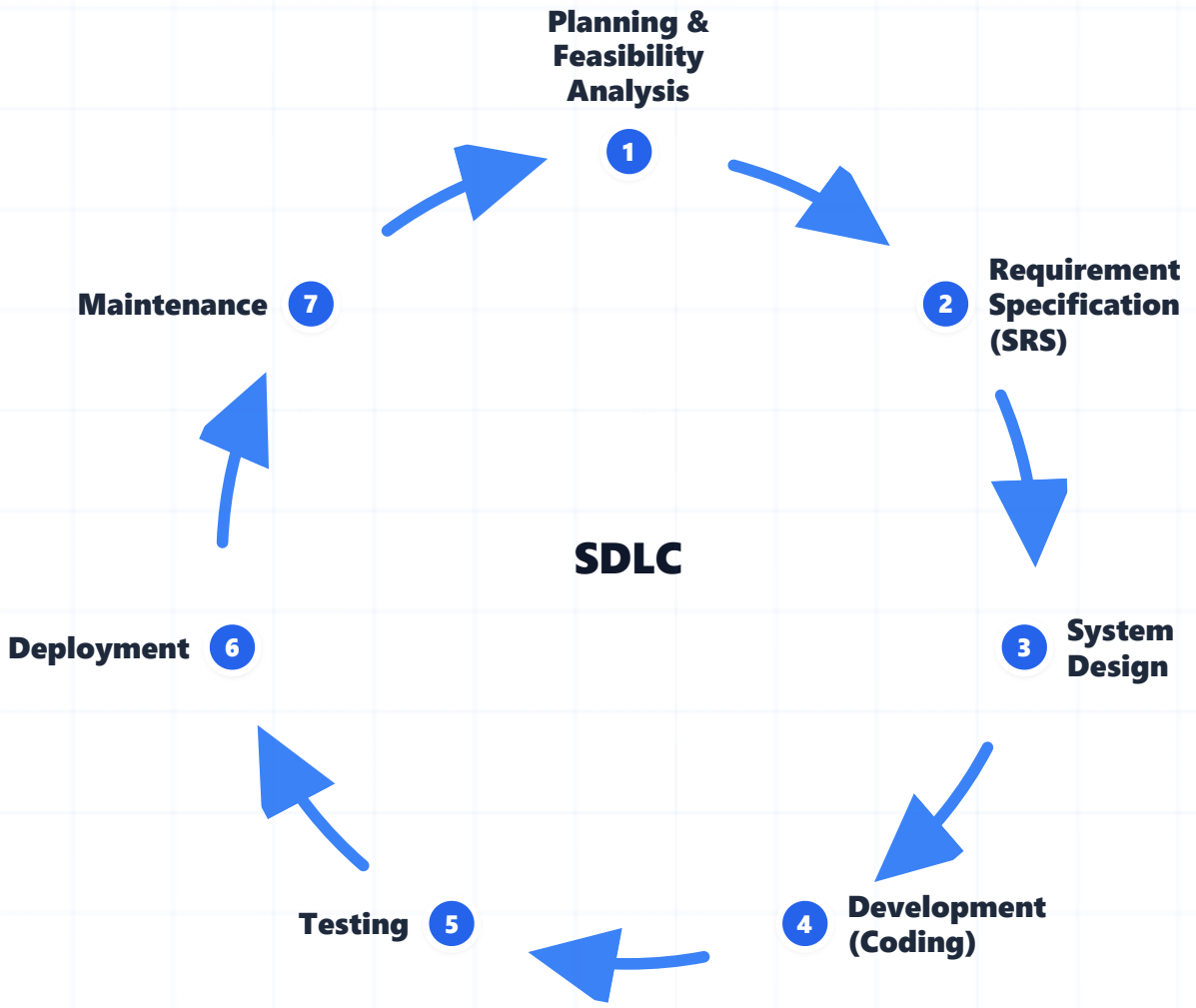
`Example: An aerospace company building flight simulation software catches an arithmetic error during the design review phase rather than post-deployment. Fixing the logic error on paper costs a fraction of an hour, whereas discovering it after release could require expensive emergency patches, recalls, or hardware tests.`

- **Better risk management:** Detect and address software defects, architectural flaws, and security vulnerabilities early in the development lifecycle.

`Example: A team developing a cloud-based customer portal performs threat modeling and vulnerability scanning during the design and implementation phases (shift-left security). By identifying an insecure authentication flaw early, they patch the vulnerability before malicious actors can exploit it in a live production release.`

1.1.2 Software development life cycle phases

The Software Development Life Cycle (SDLC) typically consists of six or seven stages, depending on the development model used.



1 Planning & Feasibility Analysis
This stage determines whether the project is technically, financially, and operationally feasible.

2 Requirement Specification (SRS)
In this stage, detailed functional and non-functional requirements are documented clearly and approved by stakeholders.

- **Activities:** Feasibility analysis, cost estimation, scheduling, resource planning
- **Output:** Project Plan, Feasibility Report
- **Key Roles:** Project Managers, Senior Engineers, Stakeholders

- **Activities:** Requirement gathering, validation, documentation
- **Output:** Software Requirement Specification (SRS)
- **Key Roles:** Business Analysts, Product Owners

3 System Design

In this stage, the approved requirements are transformed into a technical blueprint for implementation.

- **High-Level Design (HLD):** Defines system architecture, technology stack, database design, and major modules.
- **Low-Level Design (LLD):** Specifies component logic, APIs, data structures, and workflows.
- **Output:** Design Document Specification (DDS)

4 Development (Coding)

Developers build the software based on the approved design.

- **Activities:** Coding, code reviews, unit testing, version control management
- **Tools:** IDEs, version control systems, debuggers
- **Output:** Source code, executable application
- **Key Roles:** Frontend, Backend, Full Stack Developers

5 Testing

Testing ensures the software meets requirements and is free from defects before release.

Types of Testing include:

- **Unit Testing**: Verifies individual components
- **Integration Testing**: Ensures modules work together
- **System Testing:** Validates the complete system
- **User Acceptance Testing (UAT)**: Confirms business requirements are met

Output: Test cases, defect reports, and quality metrics.

6 Deployment

The tested software is released to users.

- **Activities:** Production setup, deployment, smoke testing
- **Modern Approach:** Continuous Integration and Continuous Deployment (CI/CD) pipelines for faster and reliable releases
- **Output:** Live application
- **Key Roles:** DevOps Engineers, Release Managers

7 Maintenance

Post-deployment support ensures long-term usability.

- **Activities:** Bug fixes, performance tuning, updates, feature enhancements
- **Output:** Patches, updates, new versions
- **Key Roles:** Support Engineers, Developers

fa:bell

Software Development Life Cycle (SDLC) As-Built Reference Card

- **Planning Phase**
  - *Question:* When do teams decide how to build an application and when it will be deployed by considering stakeholder needs and project feasibility?
  - *Answer:* In the **Planning** phase.
- **Analysis Phase**
  - *Question:* When should a team analyze each software requirement to create a detailed list of system specifications and use-case/data-flow diagrams?
  - *Answer:* In the **Analysis** phase.
- **Design Phase**
  - *Question:* When does a team decide on the overarching architecture, define component interactions, and utilize design patterns and prototyping tools?
  - *Answer:* In the **Design** phase.
- **Development Phase**
  - *Question:* When do teams translate design specifications into actual code while utilizing version control systems and coding standards?
  - *Answer:* In the **Development** phase.
- **Testing Phase**
  - *Question:* When does a team run comprehensive tests (such as unit, integration, system, and user testing) to assess functionality and security?
  - *Answer:* In the **Testing** phase.
- **Deployment Phase**
  - *Question:* When do teams compile a final build, prepare production environments (servers, databases, network configurations), and implement rollback strategies?
  - *Answer:* In the **Deployment** phase.
- **Maintenance Phase**
  - *Question:* When does continuous monitoring, bug fixing, updating features, and providing ongoing user support take place?
  - *Answer:* In the **Maintenance** phase.

Software Requirements Specification (SRS) Example OPEN PDF

1.1.3 SDLC management tips and best practices

Enhance the efficiency and effectiveness of your software development process by adopting various tools and techniques, such as:

- **Establishing clear communication.** Ensure all stakeholders and developers are aligned on the scope and objectives of your project to reduce the likelihood of costly reworks.
- **Implementing AI and low-code tools.** Provide innovative tools that help teams speed up development while remaining compliant and secure.
- **Creating detailed documentation.** Maintain a collection of records describing various aspects of your project to help onboard developers and meet regulatory standards.
- **Using modular designs.** Break your software design down into different components and define specifications for each one to help streamline the development process.
- **Defining coding standards.** Establish a set of strict development guidelines that help ensure the consistency and readability of your code.
- **Using design tools.** Adopt prototyping tools to efficiently model your software, validate design concepts, and gather stakeholder feedback.
- **Automating tasks and processes.** Offer automation capabilities that help speed up software development, testing, and maintenance using tools like Microsoft Power Automate.

1.2 Methods and Models: The Strategic Approach

Once the macro-level stages of the SDLC are established, organizations must choose a **development method** (or methodology) to dictate *how* they will navigate those stages. A method establishes the strategic rules, governance, and philosophy of the project. For example:

- **Traditional methods** (such as Waterfall) map the SDLC into a strict, sequential, one-way progression where each phase must be completed and signed off before moving to the next.
- **Agile methods** (such as Scrum or Extreme Programming) take those same SDLC stages and organize them into short, flexible, iterative cycles focused on incremental delivery and continuous feedback.

1.3 Processes: The Operational Execution

Finally, residing within methods and SDLC phases are individual **processes**. A process represents the specific operational workflows, tasks, and daily practices executed by team members to get work done.

While a method dictates the overall strategy, the process is the hands-on execution—such as how a developer writes code during the implementation phase, how peer reviews are conducted, or how automated builds are triggered.

2 Classical Waterfall Model

The **Waterfall methodology** is one of the earliest and most traditional software development lifecycle (SDLC) models. First formalized by computer scientist Winston Royce in a 1970 research paper, it is a **linear and sequential approach** to project management and software engineering.

True to its name, the process cascades downward through distinct, non-overlapping phases—much like a waterfall.

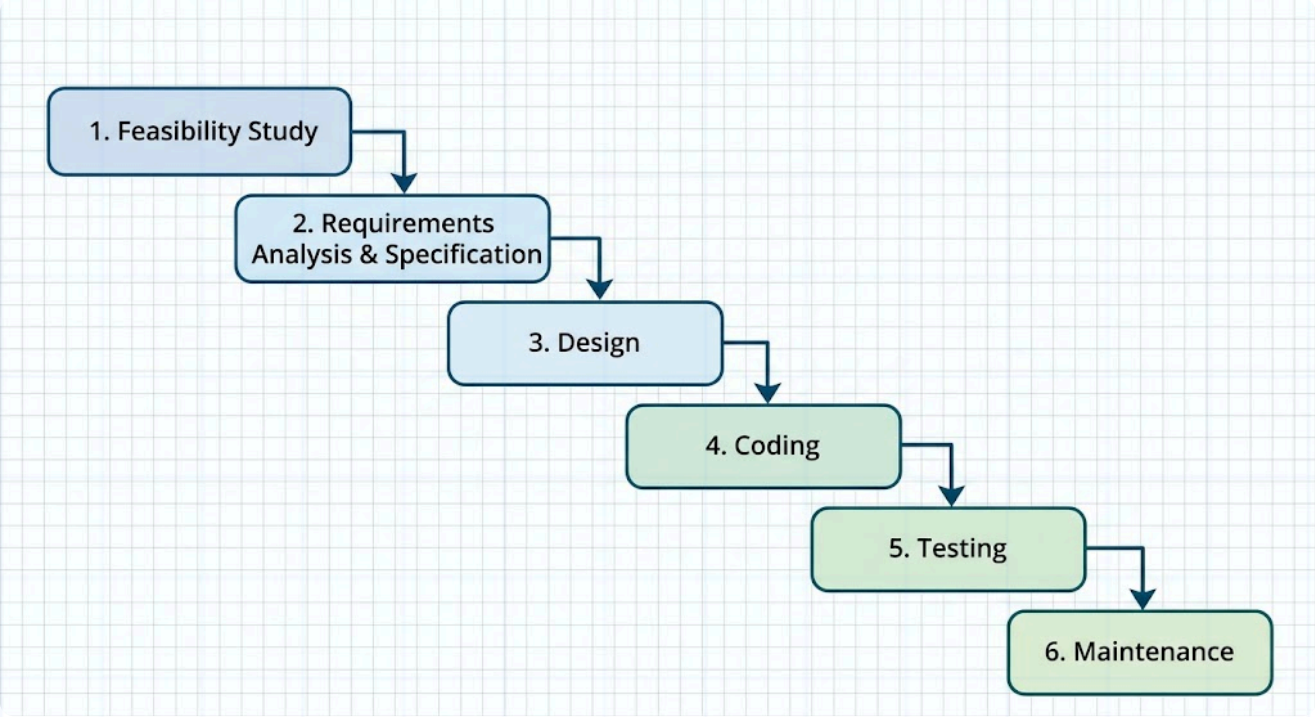


Figure 4: Waterfall methodology

2.1 Core Characteristics

- **Sequential Flow:** Progression moves strictly one way. If an issue arises later in the lifecycle, resolving it often requires stepping all the way back to an earlier phase.
- **Heavy Upfront Planning:** Scope, timelines, requirements, and resources are meticulously defined, documented, and locked down before any implementation starts.
- **Comprehensive Documentation:** The model relies on detailed system requirement specifications and design documents to serve as the blueprint for every team member.

2.2 The Standard Phases

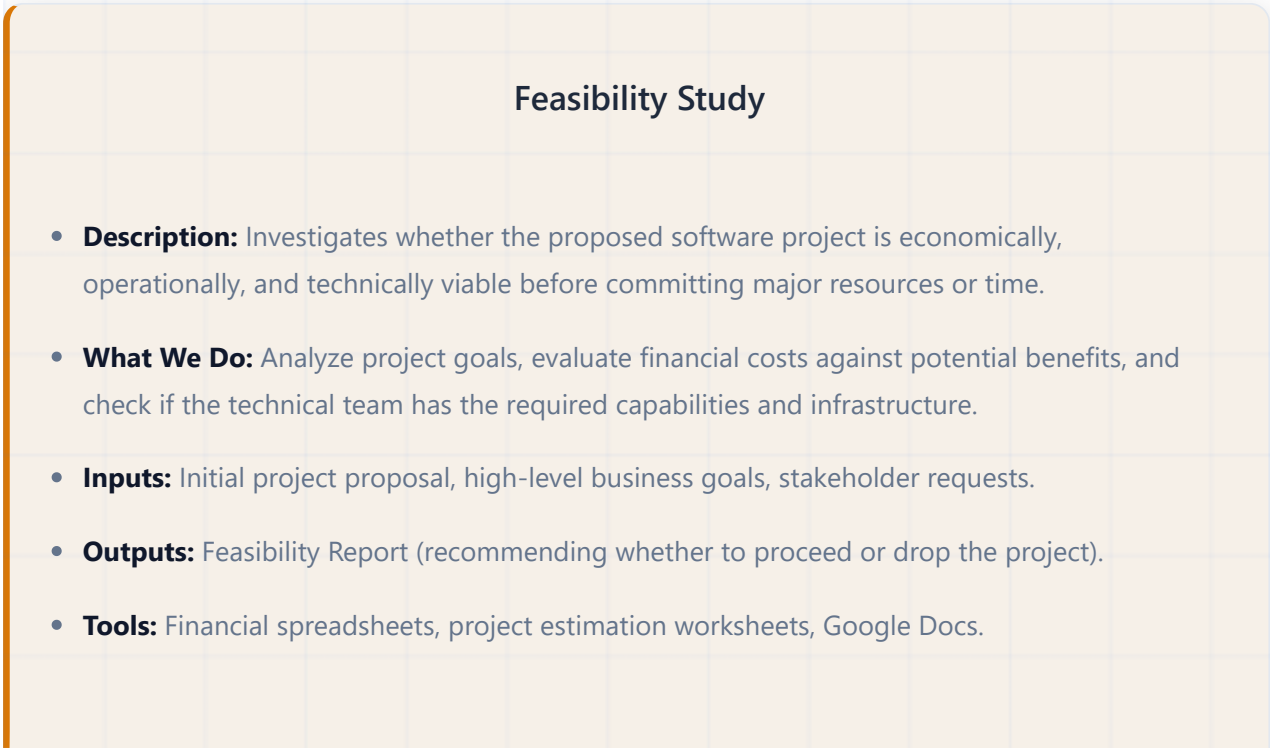
2.2.1 Feasibility study

The main aim of feasibility study is to determine whether it would be financially and technical feasible to develop the product.

- At first project managers or team leaders try to have a rough understanding of what is required to be done by visiting the client side. They study different input data to the system and output data to be produced by the system. They study what kind of processing is needed to be done on these data and they look at the various constraints on the behavior of the system.
- After they have an overall understanding of the problem they investigate the different solutions that are possible. Then they examine each of the solutions in terms of what kind of resources required, what would be the cost of development and what would be the development time for each solution.
- Based on this analysis they pick the best solution and determine whether the solution is feasible financially and technically. They check whether the customer budget would meet the cost of the product and whether they have sufficient technical expertise in the area of development.

📖 DEFINITION 2: **Feasibility Study Report**

A document generated during the feasibility study phase that outlines whether it is financially and technically viable to develop the product, providing project managers and team leaders with a preliminary understanding of the project scope, input data streams, and desired outputs before committing resources.



2.2.2 Requirements Analysis and Specification

The aim of the requirements analysis and specification phase is to **understand the exact requirements of the customer and to document them properly**. This phase consists of two distinct activities, namely

- **Requirements gathering and analysis**
- **Requirements specification**

The goal of the requirements gathering activity is to collect all relevant information from the customer regarding the product to be developed. This is done to clearly understand the customer requirements so that incompleteness and inconsistencies are removed. The requirements analysis activity is begun by collecting all relevant data regarding the product to be developed from the users of the product and from the customer through interviews and discussions. For example, to perform the requirements analysis of a business accounting software required by an organization, the analyst might interview all the accountants of the organization to ascertain their requirements. The data collected from such a group of users usually contain several contradictions and ambiguities, since each user typically has only a partial and incomplete view of the system. Therefore it is necessary to identify all ambiguities and contradictions in the requirements and resolve them through further discussions with the customer. After all ambiguities, inconsistencies, and incompleteness have been resolved and all the requirements properly understood, the requirements specification activity can start. During this activity, the user requirements are systematically organized into a **Software Requirements Specification (SRS) document**. The customer requirements identified during the requirements gathering and analysis activity are organized into a SRS document. The important components of this document are **functional requirements, the nonfunctional requirements, and the goals of implementation.**

📖 DEFINITION 3: **Software Requirements Specification (SRS)**

A formal document that captures the exact requirements of the customer, combining the outcomes of requirements gathering and analysis into a clear, comprehensive description that serves as a binding reference for the software to be developed.

Requirements Analysis and Specification

- **Description:** Gathers, analyzes, and explicitly defines all functional and non-functional requirements of the system. Once this phase concludes, requirements are frozen to prevent scope creep.
- **What We Do:** Interview stakeholders and end-users, document system constraints, and establish a clear, unambiguous behavioral contract for the software.
- **Inputs:** Feasibility report, stakeholder interviews, domain guidelines.
- **Outputs:** Software Requirements Specification (SRS) document, Use-Case diagrams.
- **Tools:** Requirements management software, Jira, Balsamiq (for wireframing), Google Docs.

2.2.3 Design

The goal of the design phase is to transform the requirements specified in the SRS document into a structure that is suitable for implementation in some programming language. In technical terms, during the design phase the software architecture is derived from the SRS document. Two distinctly different approaches are available: the traditional design approach and the object-oriented design approach.

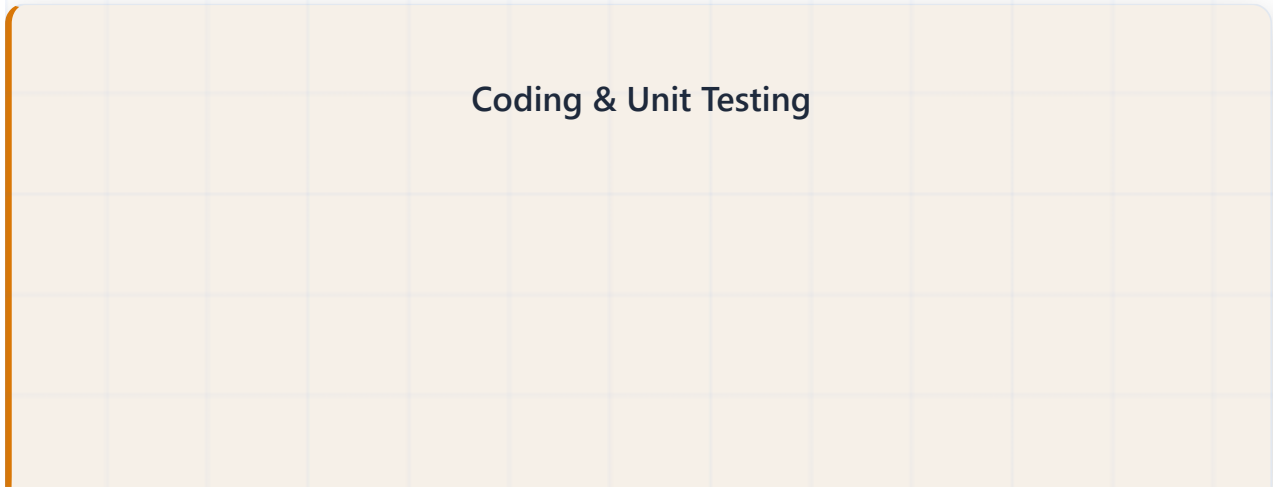
- Traditional design approach -Traditional design consists of two different activities; first a structured analysis of the requirements specification is carried out where the detailed structure of the problem is examined. This is followed by a structured design activity. During structured design, the results of structured analysis are transformed into the software design.
- Object-oriented design approach -In this technique, various objects that occur in the problem domain and the solution domain are first identified, and the different relationships that exist among these objects are identified. The object structure is further refined to obtain the detailed design.

Design

- **Description:** The goal of the design phase is to transform the requirements specified in the SRS document into a structure that is suitable for implementation in a programming language. The software architecture is derived directly from the SRS.
- **What We Do:**
    - Conduct structured analysis of requirements specification to examine the detailed structure of the problem.
    - Apply traditional design approaches (transform-centered or transaction-centered data flow design) or object-oriented design.
    - Define module coupling, cohesion, and database schemas.
- **Inputs:** Approved Software Requirements Specification (SRS) document.
- **Outputs:** Software Design Document (SDD), Entity-Relationship (ER) diagrams, data flow diagrams (DFDs), and structure charts.
- **Tools:** CASE tools, Lucidchart, Microsoft Visio, architecture blueprinting utilities.

2.2.4 Coding and unit testing

The purpose of the coding phase (sometimes called the implementation phase) of software development is to translate the software design into source code. Each component of the design is implemented as a program module. The end-product of this phase is a set of program modules that have been individually tested. During this phase, each module is unit tested to determine the correct working of all the individual modules. It involves testing each module in isolation as this is the most efficient way to debug the errors identified at this stage.



- **Description:** Translates the detailed design blueprints into executable program modules using a chosen programming language, followed immediately by developer-led unit testing of individual components.
- **What We Do:**
    - Write source code for each module according to design specifications.
    - Adhere to coding standards and guidelines.
    - Perform unit testing on individual functions and modules in isolation to check for local logic errors.
- **Inputs:** Software Design Document (SDD), module interface specifications.
- **Outputs:** Executable source code modules, passing unit test results.
- **Tools:** Integrated Development Environments (IDEs like GCC, CLion, VS Code), debuggers, unit testing frameworks (e.g., JUnit, PyTest).

2.2.5 Integration and system testing

Integration of different modules is undertaken once they have been coded and unit tested. During the integration and system testing phase, the modules are integrated in a planned manner. The different modules making up a software product are almost never integrated in one shot. Integration is normally carried out incrementally over a number of steps. During each integration step, the partially integrated system is tested and a set of previously planned modules are added to it. Finally, when all the modules have been successfully integrated and tested, system testing is carried out. The goal of system testing is to ensure that the developed system conforms to its requirements laid out in the SRS document. System testing usually consists of three different kinds of testing activities:

- α – testing: It is the system testing performed by the development team.
- β –testing: It is the system testing performed by a friendly set of customers.
- Acceptance testing: It is the system testing performed by the customer himself after the product delivery to determine whether to accept or reject the delivered product.

System testing is normally carried out in a planned manner according to the system test plan document. The system test plan identifies all testing-related activities that must be performed, specifies the

schedule of testing, and allocates resources. It also lists all the test cases and the expected outputs for each test case.

Integration & System Testing

- **Description:** Combines independently coded modules incrementally to test their cooperative behavior, culminating in comprehensive system-wide testing against the original requirements.
- **What We Do:**
  - Perform integration testing (detecting interface flaws and communication bugs between modules).
  - Execute system testing to ensure the integrated product fulfills operational and performance requirements.
  - Verify all documented test cases and log execution outputs.
- **Inputs:** Integrated source code modules, system test plan, original SRS document.
- **Outputs:** Tested software product, test case execution records, bug tracking logs.
- **Tools:** Automated testing suites, test harness frameworks, bug tracking and issue management software.

2.2.6 Maintenance

Maintenance of a typical software product requires much more than the effort necessary to develop the product itself. Many studies carried out in the past confirm this and indicate that the relative effort of development of a typical software product to its maintenance effort is roughly in the 40:60 ratios. Maintenance involves performing any one or more of the following three kinds of activities:

- Correcting errors that were not discovered during the product development phase. This is called corrective maintenance.
- Improving the implementation of the system, and enhancing the functionalities of the system according to the customer's requirements. This is called perfective maintenance.

- Porting the software to work in a new environment. For example, porting may be required to get the software to work on a new computer platform or with a new operating system. This is called adaptive maintenance.

Maintenance

- **Description:** Addresses changes, enhancements, and defect corrections required after the software product has been delivered and deployed to the customer environment. Studies confirm maintenance consumes a major portion of total product lifecycle effort.
- **What We Do:**
  - **Corrective Maintenance:** Repairing residual bugs or errors discovered post-release.
  - **Adaptive Maintenance:** Modifying the software to run in new or changed hardware/software environments.
  - **Perfective Maintenance:** Enhancing functionality or performance based on ongoing user requests.
- **Inputs:** Delivered software product, customer change requests, error/fault reports.
- **Outputs:** Software patches, version updates, updated documentation.
- **Tools:** Version control systems, patch management tools, error log monitoring systems.

2.3 Dealing with Defects in the Classical Waterfall Model

The classical waterfall model is an idealistic approach because it assumes that engineers never commit development errors during any of the life cycle phases. **But** In real-world environments, engineers frequently commit a large number of errors in almost every phase due to oversight, wrong assumptions, inappropriate technology, or communication gaps. These defects usually go unnoticed until much later in the life cycle—such as a design defect remaining undiscovered until the coding or testing phase. Once a defect is finally detected, engineers are forced to go back to the exact phase where the error originally occurred. They must then redo all the work done during that phase as well as all subsequent phases to fix the defect and its effects on the later stages.

Conclusion on Classical Execution: Because fixing errors requires this disruptive backward movement, the textbook notes that it is practically impossible to strictly follow the classical waterfall model in actual software development.

2.4 Projects Best Suited for the Waterfall Methodology

The Waterfall methodology works best for projects that are **predictable, stable, and well-understood** from the very beginning. Because it is rigid and does not easily accommodate changes midway, it thrives in environments where requirements are unlikely to shift.

Here are the key types of software projects best suited for Waterfall:

1. Highly Regulated & Safety-Critical Systems

Projects where software failure can result in loss of life, injury, or catastrophic legal/financial consequences require rigorous documentation, traceability, and strict compliance audits.

- **Examples:** Avionics/flight control software, medical device embedded systems, nuclear power plant monitoring software, and automotive braking systems.
- **Why it fits:** Every single requirement, design decision, and test case must be meticulously documented and verified before release to pass strict government or industry certifications.

2. Fixed-Scope, Small-to-Medium Enterprise Projects

Projects where the client and development team have a crystal-clear, unchanging vision of what the final product should do before a single line of code is written.

- **Examples:** A standardized inventory tracking tool for a local warehouse, an internal payroll calculator built on unchanging tax laws, or a fixed-function corporate landing page.
- **Why it fits:** With a rigid scope and zero ambiguity, the project can move smoothly from requirements to maintenance without facing unexpected change requests.

3. Government, Public Sector, & Defense Contracts

Large-scale public tenders where budgets, timelines, and deliverables must be locked down legally through binding contracts before development begins.

- **Examples:** National ID database systems, e-governance portals, or military logistics management platforms.
- **Why it fits:** Public sector projects require predictable budgeting, fixed timelines, and comprehensive historical documentation that lawmakers and auditors can review step-by-step.

4. Projects Utilizing Mature, Stable Technologies

Projects built using established, long-standing tech stacks where the development team already possesses deep expertise and knows every technical limitation in advance.

- **Examples:** Upgrading an established legacy system or migrating an existing relational database schema to a newer server using tried-and-tested frameworks.
- **Why it fits:** Because there are no experimental technologies or unknown technical roadblocks, the design and implementation phases can be accurately estimated upfront.

3 The Iterative Waterfall Model

To overcome the major shortcomings of the classical waterfall model, we come up with the iterative waterfall model. Here, we provide feedback paths for error correction as & when detected later in a phase.
Though errors are inevitable, but it is desirable to detect them in the same phase in which they occur. If so, this can reduce the effort to correct the bug. The advantage of this model is that there is a working model of the system at a very early stage of development which makes it easier to find functional or design flaws. Finding issues at an early stage of development enables to take corrective measures in a limited budget.

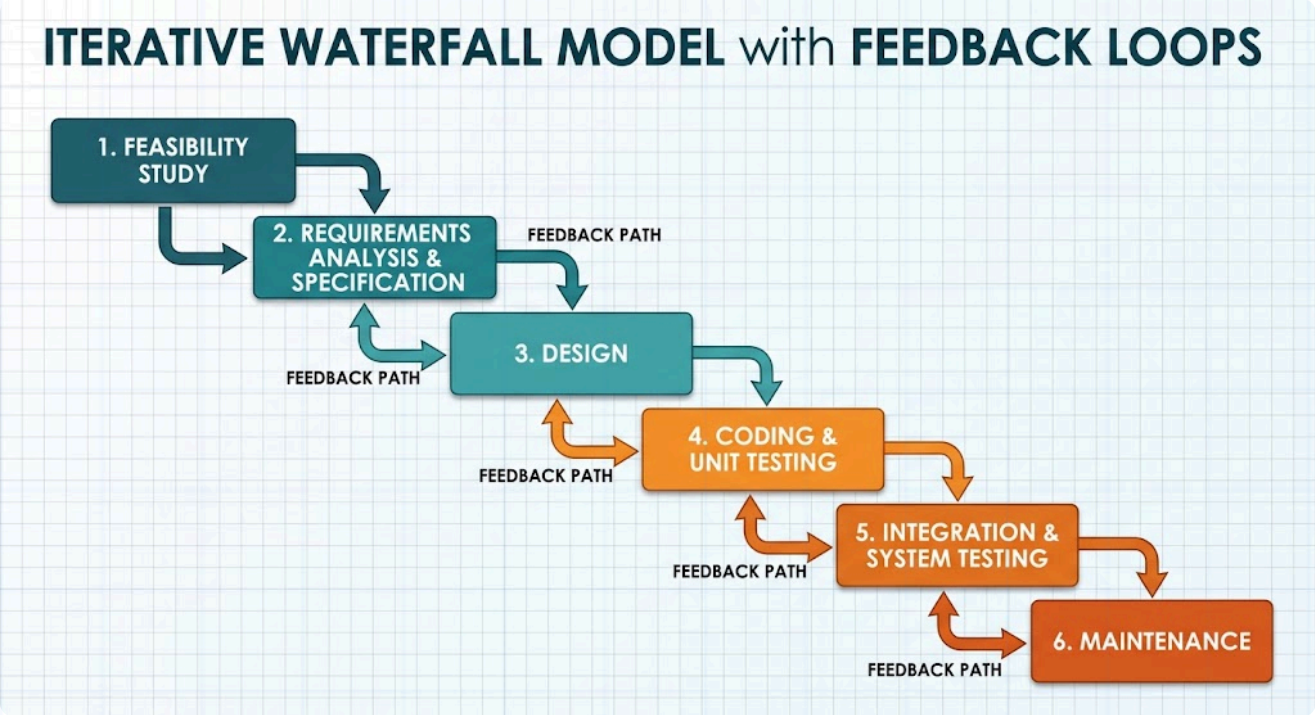


Figure 5: The Iterative Waterfall Model

3.1 Overcoming Classical Limitations: The Iterative Waterfall Model

- **Built-in Feedback Paths:** Unlike the classical waterfall model which forces a strict, non-reversible linear flow, the iterative model introduces feedback paths from downstream phases (like testing or coding) back to upstream phases (like design or requirements).
- **Handling Late-Discovered Defects:** When errors or design flaws are uncovered during testing, engineers can easily loop back to the preceding phases to correct them without breaking the entire development framework.
- **Iterative Refinement:** It accommodates the reality that engineers make mistakes due to oversight or changing needs by allowing the software to be refined and re-evaluated through multiple passes or iterations.
- **Practical Viability:** By acknowledging that requirements and designs evolve and that errors will inevitably be caught late, the iterative waterfall model provides a practical, working framework for real-world software engineering environments.

4 The Prototyping Model

The **Prototyping Model** (often referred to as the Prototyping Approach or Prototyping-based methodology) is designed specifically to address the major weakness of the classical waterfall model: **difficulty in fully understanding user requirements upfront** and the risk of building the wrong system.

4.1 Core Concept of the Prototyping Method

Instead of locking down all requirements in an exhaustive SRS document before any design or coding begins (as the classical waterfall model demands), the prototyping approach starts by building a **quick, working model (prototype)** of the system.

1. **Initial Requirement Gathering:** Collect basic, high-level requirements.
2. **Quick Design & Prototype Construction:** Build a scaled-down version or mockup of the system (focusing primarily on the user interface and core visible interactions).
3. **User Evaluation & Feedback:** Present the prototype to the customer/users. They interact with it, discover what they actually like or dislike, and provide concrete feedback.
4. **Refining Requirements:** Update and refine the system specifications based on real user feedback.
5. **Engineering the Final Product:** Once the prototype is refined and approved, the final production-quality system is built using standard development phases.

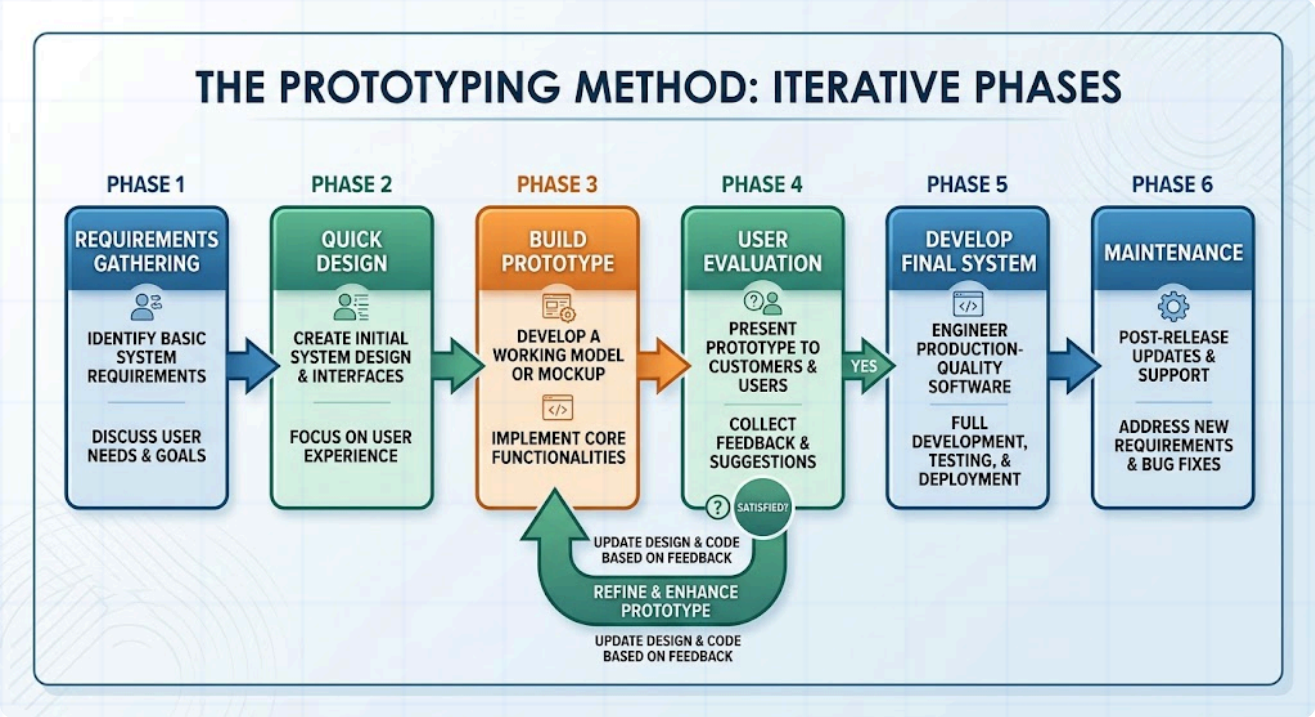


Figure 6: Prototyping Method

4.2 Need for a prototype in software development

There are several uses of a prototype. An important purpose is to illustrate the input data formats, messages, reports, and the interactive dialogues to the customer. This is a valuable mechanism for gaining better understanding of the customer's needs:

- how the screens might look like
- how the user interface would behave
- how the system would produce outputs

Another reason for developing a prototype is that it is impossible to get the perfect product in the first attempt. Many researchers and engineers advocate that if you want to develop a good product you must plan to throw away the first version. The experience gained in developing the prototype can be used to develop the final product. A prototyping model can be used when technical solutions are unclear to the development team. A developed prototype can help engineers to critically examine the technical issues associated with the product development. Often, major design decisions depend on issues like the response time of a hardware controller, or the efficiency of a sorting algorithm, etc.
In such circumstances, a prototype may be the best or the only way to resolve the technical issues.
A prototype of the actual product is preferred in situations such as:

- User requirements are not complete
- Technical issues are not clear

4.3 Integration of Prototyping in the Iterative Requirements Phase

While the classical waterfall model relies on a strict, linear progression, practical engineering environments frequently encounter scenarios where customer requirements are vague, ambiguous, or technically risky. To bridge the communication gap before freezing or finalizing the Software Requirements Specification (SRS), software teams can adopt a hybrid, evolutionary approach by integrating rapid prototyping directly into the Requirements Analysis phase of an iterative lifecycle.

Practical Workflow for Requirements Prototyping

When uncertainties arise during initial stakeholder elicitation, the requirements phase incorporates the following evolutionary loops:

- **Identifying Mismatches Early:** If developers and customers suspect a fundamental misunderstanding regarding system features, user workflows, or interface designs during requirements gathering, a rapid, working mockup is constructed.
- **Proving Concepts:** The customer directly interacts with the prototype to validate initial assumptions, test usability parameters, and clarify their actual operational needs.
- **Returning to Requirements:** Once empirical feedback is gathered through the prototype, the team refines the functional specifications, updates the SRS document, and resolves ambiguities before formally moving forward into the design and coding phases.

5 The Incremental Model

The **incremental model** is a software development approach in which the product is designed, implemented, and tested piece by piece, with a little more functionality added in each cycle until the complete system is ready. The product is considered finished only when it satisfies all of its requirements.

> 📖 DEFINITION 4: **Incremental Model**
>
> adaptive development approach in which the deliverable is produced successively, adding functionality until the deliverable contains the necessary and sufficient capability to be considered complete.

Here is the core idea in plain language:

- The complete system is broken down into several self-contained modules or **builds**.
- The highest priority requirements are tackled **first**.

- Each build goes through its own mini life cycle (requirements, design, coding, testing) and is then delivered as a working, usable version.
- The customer uses each delivered build, gives feedback, and that feedback shapes the next build.
- The process repeats until the full product is delivered.

> **Note**
>
> This approach combines the discipline of the waterfall model (each build still follows a linear sequence of phases) with the iterative philosophy of prototyping (learn and refine as you go).

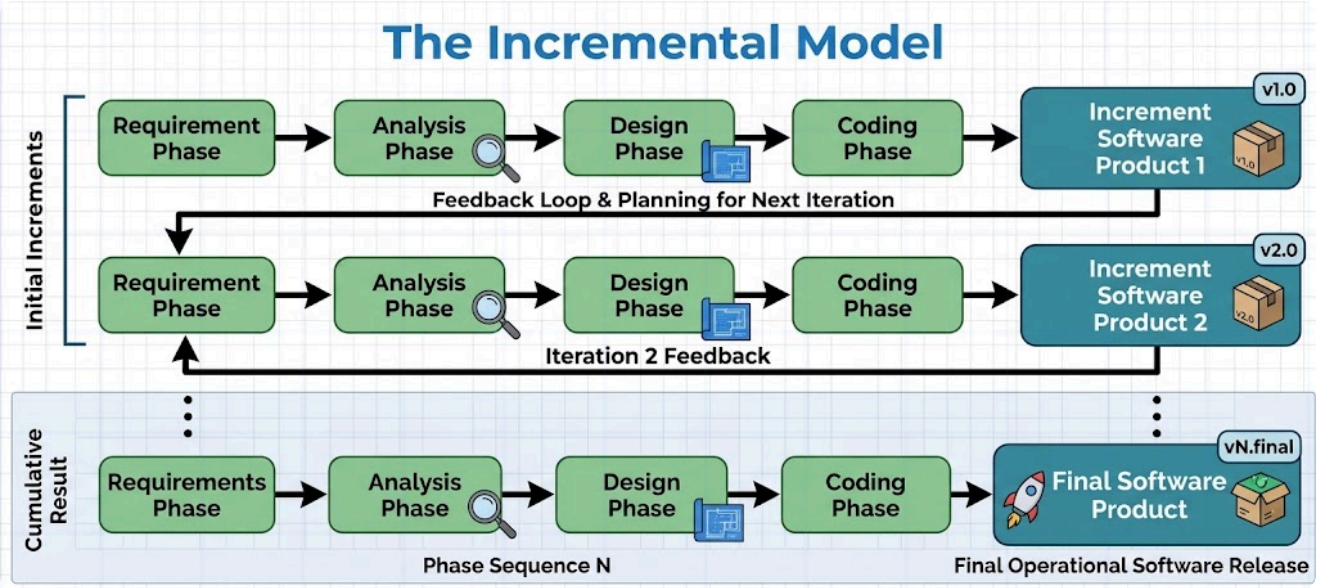


Figure 7: The Incremental Model

5.1 When to Use the Incremental Model

The incremental model is a strong fit when:

- **Requirements are mostly known, but expected to evolve.** The major requirements must be defined up front so the system can be sensibly split into increments, but some details can grow over time.
- **Time to market is critical.** You need a usable product in front of users fast, and you want to keep shipping value while the rest is built.
- **The project is large or complex.** Breaking a big system into builds makes it manageable and reduces the chance of one catastrophic failure.
- **Early revenue or feedback matters.** A partial product that works is worth more than a "complete" product that arrives too late.

- **The team is adopting new technology.** Riskier or unfamiliar parts can be tackled in early increments so problems surface while they are still cheap to fix.
- **Budget is limited or phased.** Costs can be spread across increments instead of requiring one huge upfront investment.
- **Stakeholders can stay engaged.** The model works best when customers review each build and provide feedback.

5.2 Incremental vs Iterative vs Waterfall: What is the Difference?

These three terms are often used interchangeably, but they are not the same. Getting this distinction right is one of the most common interview questions on SDLC models.
Incremental means you **add** whole, usable slices of functionality one at a time. Think of the food delivery example: each release added a complete, working feature (payment, then tracking, then personalization). In agile terms these are called **vertical increments**, because each new version is a usable product that builds on the previous one.

Iterative means you **refine** the same functionality through repeated cycles. You build a rough version, learn from it, and improve it in the next cycle. An iterative model often starts from an MVP (minimum viable product) and repeatedly improves it.

Waterfall is strictly sequential. All requirements are gathered up front, then design, then coding, then testing, and the customer sees the finished product only at the very end.

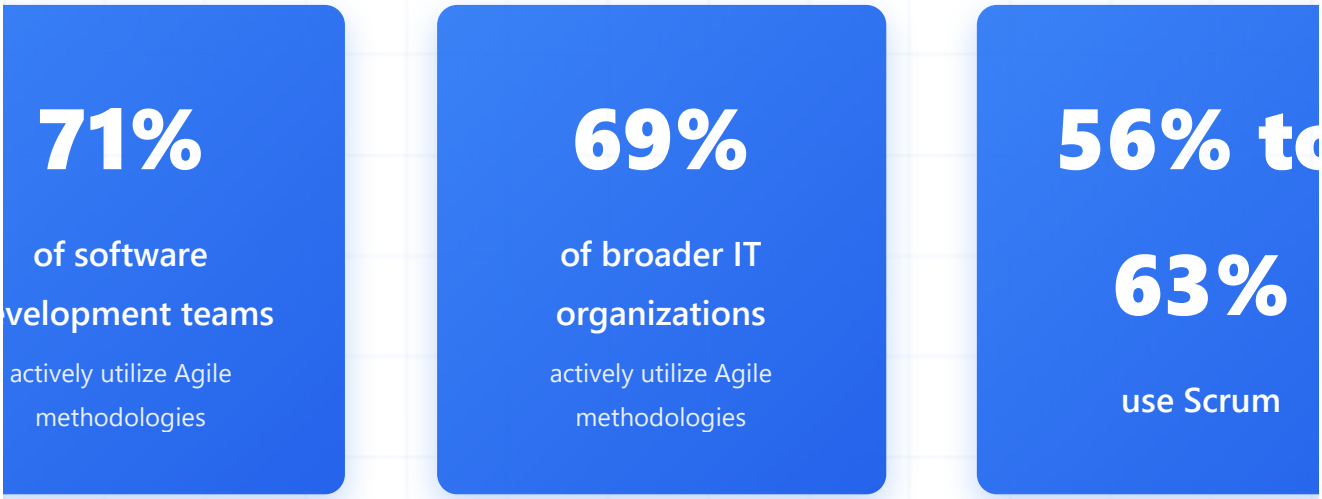
In practice, real projects are usually **both** iterative and incremental: each cycle adds a new slice (incremental) *and* improves earlier slices (iterative). The Agile Alliance notes that it is hard to imagine an incremental approach that is not at least somewhat iterative, even though the two concepts are distinct.

Table 1: *Incremental vs Iterative vs Waterfall: What is the Difference?*

Model	Core idea	Customer sees product	Change handling	Best for
Waterfall	One linear sequence of phases	Only at the end	Weak, changes are costly	Fixed, well understood requirements
Incremental	Add complete, usable slices one by one	Early and continuously	Good	Large projects that need early delivery
Iterative	Refine the same product through repeated cycles	Early (rough versions)	Good	Requirements that are unclear at the start
Agile	Incremental + iterative, with close collaboration and short sprints	Every sprint	Excellent	Fast moving products with active stakeholders

6 Agile Model

The **Agile Model** is an SDLC model focused on iterative software product development. The Agile paradigm is an incremental model in which software is built in quick increments. The most significant aspect of the Agile model is determining the project scope, requirements, number, and duration of iterations at the start of the development process—**making it the ideal choice when requirements are unclear, evolving, or expected to change over time through continuous feedback.**



6.1 Why do organizations choose Agile?

According to HP, out of 403 organizations that have primarily adopted Agile, the following percentage of respondents agreed with the following statements about Agile development:

- 54%: Enhances collaboration between teams that don't usually work together.
- 52%: Increases the level of software quality in organizations.
- 49%: Results in increased customer satisfaction.
- 43%: Shortens time to market.
- 42%: Reduces cost of development.

6.2 When to Use the Agile Model?

The Agile Model is best utilized under specific project conditions where flexibility, continuous feedback, and speed to market are top priorities.

Here are the key scenarios when you should use the Agile model:

- **Evolving Requirements:** Ideal when project requirements are expected to change, grow, or evolve over time rather than being completely fixed upfront.
- **Frequent Feature Additions:** When stakeholders or users need regular, incremental additions of new features at a minimal cost.
- **Rapid Time-to-Market:** Suitable when there is a strong business need to deliver usable software quickly and iterate continuously, rather than waiting for a massive final release.
- **Complex and Innovative Projects:** Effective for complex, cutting-edge projects where an adaptive, trial-and-error approach helps teams discover the best technical and functional path forward.
- **Active Stakeholder Engagement:** Works best when customers, product owners, and end-users are readily available to review working increments and provide continuous feedback.
- **Cross-Functional Teams:** Well-suited for collaborative teams where members share ownership and work closely together across development, testing, and deployment.

6.3 Agile Manifesto and Core Foundations

The Agile Manifesto defines a flexible approach to product and software development through four core values, twelve guiding principles, and various execution practices.

6.3.1 4 Core Values

The foundation of the Agile philosophy is built upon four core values. These values form the bedrock of the Agile Manifesto, providing the philosophical guidance for teams that choose an Agile approach. While all parts of the manifesto have value, Agile prioritizes the items on the left side of each statement over those on the right.

Right-Side Priorities (Valued, but not prioritized)	**OVER**	**Left-Side Priorities (Given top priority)**
Rigid **Processes and Tools**	→	**Individuals and Interactions**
Comprehensive **Documentation**	→	**Working Software**
Strict **Contract Negotiation**	→	**Customer Collaboration**
Rigidly **Following a Plan**	→	**Responding to Change**

A Closer Look at Each Value:

1. **Individuals and Interactions over Processes and Tools**
   - **The Right (Traditional):** In traditional project management, a significant amount of emphasis is placed on established processes, methodologies, and the tools used to carry them out. The assumption is that if everyone follows the process, the project will be a success.
   - **The Left (Agile):** Agile recognizes that it is people—their talent, creativity, and ability to communicate—who ultimately create successful products. While processes and tools are still necessary, they are only useful if they enable individuals to work together effectively. A great team can overcome a poor process, but a great process will not fix a poor team.
2. **Working Software over Comprehensive Documentation**
   - **The Right (Traditional):** Traditional software development methods often involve large, initial requirements definition phases, detailed specification documents, and complex architectural diagrams. Success is often measured by whether the "done" paperwork is completed.
   - **The Left (Agile):** Agile teams believe that the most valuable progress is working software that customers can actually use. They produce minimal, essential documentation (such as user stories) and focus their primary energy on coding, testing, and getting working versions of the software in front of users as frequently as possible. Documents may define what *should* be, but software is what *is*.
3. **Customer Collaboration over Contract Negotiation**
   - **The Right (Traditional):** This viewpoint treats the development process like a transactional procurement. Teams write a complex contract at the start and then use that rigid document as the primary tool for communication with the customer.

- **The Left (Agile):** Agile shifts the relationship with the customer from one of a rigid transaction to one of joint creation and collaboration. Successful teams invite customers into the process early and often to get their continuous feedback, ensure they are on the right track, and work together to deliver the most valuable features first. The contract should set up a structure for successful work, not create barriers to it.

4. **Responding to Change over Following a Plan**

- **The Right (Traditional):** Success in traditional methods is often defined as following a fixed, long-term plan created at the start of a project, and any divergence is viewed negatively.
- **The Left (Agile):** Agile teams embrace the fact that requirements will naturally change as the customer's business evolves and as they learn more from their initial releases. Agile processes are designed to be flexible and adaptive, prioritizing the ability to respond to and learn from change rather than resisting it simply because it wasn't in the original, outdated plan. They plan in small chunks, with a clear understanding that their knowledge is partial and will grow.

> ⓘ **Note**
>
> It's important to understand that Agile does not *completely reject* the items on the right. There is still a place for well-defined processes, appropriate documentation, clear contract terms, and a general plan for where the project is headed. However, in the everyday work of software development, Agile forces a different choice: **When a conflict arises, prioritize the items on the left.**

6.3.2 12 Guiding Principles

The values are supported by twelve operational tenets outlined in the Agile Manifesto:

1. **Customer satisfaction:** Achieve priority through early and continuous delivery of valuable software.
2. **Welcome change:** Harness changing requirements to give the customer a competitive advantage, even late in development.
3. **Frequent delivery:** Deliver working software frequently, ranging from a couple of weeks to a couple of months, preferring shorter timescales.
4. **Daily collaboration:** Business people and developers must work together daily throughout the project.
5. **Motivated individuals:** Build projects around trusted, supported, and motivated individuals.
6. **Face-to-face conversation:** Use direct conversation as the most efficient and effective way to convey information.

7. **Working software as a progress measure:** Measure primary progress through functional, working software.
8. **Sustainable development:** Maintain a constant, sustainable pace indefinitely for sponsors, developers, and users.
9. **Technical excellence:** Maintain continuous attention to good design and technical craftsmanship.
10. **Simplicity:** Maximize the amount of work not done—keep things as simple as possible.
11. **Self-organizing teams:** Allow the best architectures, requirements, and designs to emerge from self-organizing teams.
12. **Regular reflection:** Have the team regularly reflect on how to become more effective, then tune and adjust behavior accordingly.

6.3.3 From Agile Theory to Practical Frameworks

While the Agile Manifesto establishes the foundational mindset and values, it intentionally remains broad and high-level. To bridge this gap between abstract theory and real-world execution within the Software Development Life Cycle (SDLC), organizations adopt concrete agile frameworks and methods to turn these principles into everyday workflows:

- **Scrum:** Working in fixed-length iterations called sprints, holding daily stand-ups, sprint planning sessions, and retrospectives, backed by defined roles like the Product Owner and Scrum Master.
- **Kanban:** Visualizing work items on a board, limiting work-in-progress (WIP), and managing continuous flow across columns to prevent bottlenecks.
- **Extreme Programming (XP):** Using rigorous engineering practices like pair programming, continuous integration, test-driven development (TDD), and frequent refactoring to ensure high code quality.
- **Backlog Grooming / Refinement:** Continuously prioritizing and updating the product backlog based on evolving user feedback and changing requirements.

6.4 Extreme Programming

Extreme Programming (XP) is a lightweight method for developing software with constantly evolving or unclear requirements, such as business-focused systems (Type B), according to the classification previously. Like any agile method, XP embodies characteristics like short iterative development cycles, reduced emphasis on comprehensive documentation, incremental design, and development in small teams.

> 📖 DEFINITION 5: **Extreme Programming (XP)**

> A disciplined Agile software development framework focused on adaptability, customer satisfaction, and high-quality code.

6.4.1 Extreme Programming (XP) Phases & Workflow

The Extreme Programming life cycle operates on a continuous feedback loop that scales from multi-month planning horizons down to micro-level coding tasks in seconds.

1. Release Planning (Months)

- **What happens:** The customer and team decide on the scope of the next release by picking high-priority user stories. This defines *what* features the system will deliver by a specific target date (typically a few months out).
- **Key Element - User Stories:** Requirements are gathered and documented as lightweight **user stories** rather than a monolithic specification document. These stories act as the functional requirements, capturing what features the system must perform and continuously evolving as the project progresses.
- **Goal:** Prioritize high-value features and establish a target delivery timeline spanning a few months.

2. Iteration Planning (Weeks)

- **What happens:** A release is broken down into multiple short iterations (usually lasting 1 to 3 weeks). At the start of each iteration, the development team plans out the specific tasks required to implement that cycle's portion of the release.
- **Key Element - Iteration Backlog:** At the start of each iteration, the development team plans out the specific tasks required to implement that cycle's features.

3. Acceptance Testing & Stand-Up (Days to One Day)

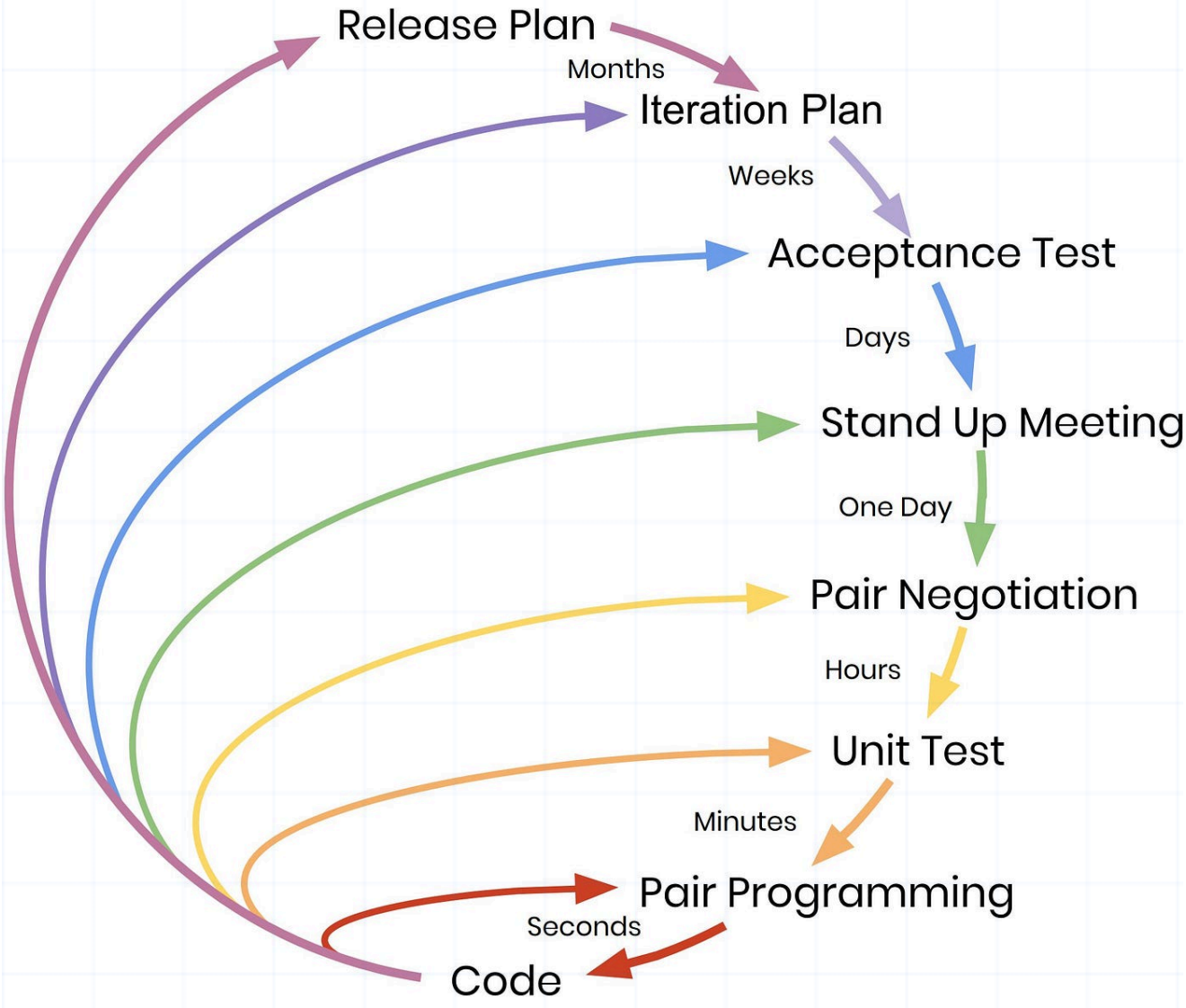
- **What happens:** As features take shape, they are validated against business requirements.
- **Key Elements:**
  - **Acceptance Tests:** Functional tests written by the customer (or testers) to verify that the system satisfies the user stories. A release is only considered finished and ready for customer approval once these tests pass.
  - **Stand-up Meetings:** Daily syncs for the team to align on daily progress and blockers.

4. Pair Negotiation & Unit Testing (Hours to Minutes)

- **What happens:** Developers dive into technical execution, leveraging tight feedback loops.
- **Key Elements:**
  - **Pair Programming:** Two developers working together at one workstation to write and continuously review code, boosting code quality and minimizing defects.
  - **Test-Driven Development (TDD):** Developers write automated **unit tests** *before* writing the actual code. Every piece of code must pass all unit tests before moving forward.

5. Coding & Small Releases (Seconds)

- **What happens:** The micro-level code is written, integrated frequently into a shared repository, and delivered in frequent, small production increments.



6.4.2 XP Core Values (Beyond the General Agile Manifesto)

While general Agile has 4 values, XP explicitly relies on six specific human and cultural values to guide day-to-day work(how teams operate in an XP environment):

- **Communication:** Ensuring developers, customers, and managers talk constantly to avoid misunderstandings.
- **Simplicity:** Doing "the simplest thing that could possibly work" and avoiding over-engineering (YAGNI - *You aren't going to need it*).
- **Feedback:** Getting rapid signals from automated tests, customer reviews, and system deployment to catch mistakes early.
- **Courage:** Telling the truth about estimates, discarding code that doesn't work, and being willing to refactor design.
- **Respect:** Valuing every team member's contribution and caring about sustainable work.
- **Quality of Life:** Ensuring developers are not overworked, preventing burnout, and maintaining a sustainable 40-hour work week.

6.4.3 Extreme Programming (XP) Practices

While general Agile frameworks provide high-level principles and management structures, Extreme Programming (XP) is uniquely defined by a set of concrete, rigorous engineering and organizational techniques. **The practices are categorized into how teams collaborate, write code, and deliver software.**

6.4.3.1 Process Practices

These practices govern how requirements are captured, planned, and delivered in short cycles:

- Customer Representative: Embedding an on-site domain expert within the team who defines requirements and remains constantly available to answer questions.
- User Stories: Lightweight, concise requirements (typically two to three sentences written on cards) focusing on core features from a user perspective.
- Iterations & Releases: Organizing development into short, fixed cycles (iterations lasting 1 to 3 weeks) grouped into broader releases spanning a few months.
- Release & Iteration Planning: Collaborative planning sessions—such as Planning Poker using Fibonacci story points—to estimate effort and map user stories to iterations.
- Slack: Intentionally programming buffer tasks into iterations to absorb unexpected delays, prevent team burnout, and allow time for learning or exploration.

6.4.3.2 Programming Practices

XP introduced revolutionary engineering techniques designed to maintain high code quality, short feedback loops, and robust architectural health from the project's inception:

- Incremental Design: Avoiding Big Design Up Front (BDUF) in favorable of continuous design evolution, guided by principles like "Do the simplest thing that could possibly work" and YAGNI ("You aren't going to need it").
- Pair Programming: Two developers working side-by-side at a single monitor—swapping roles between a Driver (controlling the keyboard) and a Navigator (reviewing code and guiding strategy)—to catch bugs early and spread team knowledge.
- Test-Driven Development (TDD): Writing automated unit tests before implementing the actual functional code to ensure clarity of interface and high test coverage.
- Automated Builds: Fully automating system builds so executables are generated without manual developer intervention.
- Continuous Integration (CI): Frequently merging code changes (ideally daily) into a shared repository backed by automated testing to prevent "integration hell" and catch compilation or logic errors immediately.
- Collective Code Ownership: Permitting any developer to modify any part of the codebase as needed, eliminating individual code silos.

6.4.3.3 Project Management Practices

These practices focus on team sustainability, working conditions, and financial alignment:

- Working Environment: Promoting small, dedicated teams (ideally fewer than 10 developers) working a sustainable 40-hour schedule to protect physical and mental well-being and prevent high employee turnover.
- Negotiated Scope Contracts: Utilizing open-scope or hourly contracts based on continuous customer collaboration rather than rigid, fixed-scope contracts that force teams to compromise software quality under penalty clauses.
- Process Metrics: Tracking key performance indicators such as the number of bugs detected in production and the time interval between project kickoff and its first financial returns.

6.4.4 End-to-End Example: Building the Q&A Forum with XP

> **EXAMPLE 1:**
>
> To fully understand how Extreme Programming (XP) works in practice, let's walk through the complete lifecycle of our Q&A forum project, starting from the very first day and following the correct XP sequence.
>
> ✅ Show Solution

Step 1: Establishing the Project Cadence (The Setup)

Before any code is written or stories are gathered, the team and management agree on the project's timeboxes.

- **Iteration Length:** 2 weeks (10 working days).
- **Release Length:** 4 weeks (comprising two 2-week iterations).
- **Goal for Release 1:** Deliver a functional, basic version of the forum that internal beta testers can use.

> ⓘ **Iteration vs. Release**
>
> In XP, it is common to confuse *iterations* and *releases* because both involve delivering working software in fixed timeboxes. However, they serve entirely different purposes in the project lifecycle.
>
> Think of it like producing a television show: an **iteration** is like filming a single episode, while a **release** is like bundling several episodes together to premiere a full season to the audience.

Feature	Iteration	Release
Duration	Short and fixed (e.g., 1 to 3 weeks).	Longer, comprising multiple iterations (e.g., 1 to 3 months).
Primary Focus	**Execution:** Breaking down stories into technical tasks, coding, testing, and integration.	**Planning & Value:** Grouping features together to deliver a cohesive, meaningful milestone to the business or users.
Who Drives It?	**The Development Team:** They own the task breakdown, daily assignments, and technical execution.	**The Customer Representative:** They own the prioritization, deciding *which* stories go into the release to maximize business value.
Planning Level	**Micro-planning:** "How will we build these specific stories? Who does what task?"	**Macro-planning (The Planning Game):** "What features do we need to deliver a usable version of the product by this date?"
Output	A set of completed, tested user stories integrated into the main codebase.	A stable, cohesive version of the software that is ready to be shown to stakeholders, beta testers, or deployed to production.

Step 2: Gathering User Stories

The Customer Representative (acting as the voice of the business/users) brainstorms the core features needed for Release 1 and writes them down as user stories. They present a list of 8 stories to the development team:

1. Register user
2. Post questions
3. Post answers
4. Opening screen
5. Gamify questions/answers
6. Search questions/answers
7. Add tags
8. Comment on questions/answers

Step 3: Estimating Effort (Planning Poker)

The developers must now estimate the complexity of these 8 stories. They gather in a room with the Customer Representative for a **Planning Poker** session.

Example Scenario: Estimating "Post Questions"

- The Customer reads the story: *"As a logged-in user, I want to be able to post questions. Since it's a programming forum, questions may incorporate code blocks, which must be presented in a differentiated layout."*
- **First Vote:** The developers reveal their cards. Developer A plays a **3**, but Developer B plays an **8**.
- **Discussion:** The team discusses the discrepancy. Developer B points out that standard text input is easy (3 points), but building a secure, syntax-highlighted layout for multi-line code blocks requires integrating a specific Markdown library and writing custom CSS, which is much harder.
- **Second Vote:** Acknowledging the complexity of the code blocks, the team re-votes. Everyone agrees on **5 story points**.

This process is repeated for all 8 stories until every story has a Fibonacci point value attached to it.

> ⓘ **What Exactly Are We Estimating?**
>
> When developers assign story points during Planning Poker, they are not estimating time (hours or days). Instead, they are estimating the **relative size** of the story. This "size" is a composite score based on three key factors:
>
> 1. **Complexity:** How difficult is the logic or architecture?
> 2. **Effort:** What is the sheer volume of work required?
> 3. **Risk/Uncertainty:** How many unknowns or potential roadblocks exist?

Step 4: Release Planning (The Planning Game)

Now the team must figure out how much work they can actually do. Based on past experience or initial capacity planning, the team agrees their **Velocity** is **21 story points per iteration**.

The Customer Representative now takes control of prioritization. They want the most critical features first. They look at the estimates and group the stories into iterations, ensuring no iteration exceeds the 21-point velocity limit.

- **Iteration 1 (21 points):** Register user (8), Opening screen (5), Post questions (5), Post answers (3). *Rationale: You can't have a forum without users, a homepage, and the ability to ask/answer.*
- **Iteration 2 (21 points):** Gamify (5), Search (8), Add tags (5), Comment (3). *Rationale: These are secondary features that enhance the core experience.*

Release planning is now complete. The team knows exactly what they are building for the next month.

Story	Story Points	Iteration	Release
Register user	8	1	1
Post questions	5	1	1
Post answers	3	1	1
Opening screen	5	1	1
Gamify questions/answers	5	2	1
Search questions/answers	8	2	1
Add tags	5	2	1
Comment on questions/answers	3	2	1

Step 5: Iteration Planning (Breaking Down the Work)

It is Monday morning, Day 1 of Iteration 1. The team gathers for **Iteration Planning**. They look at the 4 stories assigned to this iteration. They don't just start coding; they break the stories down into technical **tasks** that take a few days or less to complete.

They start with the **"Post questions"** story (5 points). Together, they decompose it into the following tasks:

1. **Design:** Web interface layout and CSS for code blocks. (Assigned to Alice, 1 day)
2. **Database:** Install DB, design and create `questions` tables. (Assigned to Bob, 1 day)
3. **Data Access:** Implement the data access layer. (Assigned to Bob, 1 day)
4. **Framework:** Install and test the web server framework. (Assigned to Charlie, 1 day)
5. **Control Layer:** Implement endpoints to register, remove, and update questions. (Assigned to Charlie, 2 days)
6. **Frontend Integration:** Implement the final web interface connecting to the backend. (Assigned to Alice, 2 days)

Adding Slack: To ensure they don't burn out and have a buffer for the unknown, the team adds a **Slack Task**: *Research and document best practices for preventing Cross-Site Scripting (XSS) in user-submitted code blocks.* (Estimated: 2 days, assigned to Charlie).

Step 6: Execution and Adapting to Reality

The iteration begins. Developers pair-program, write tests first (TDD), and communicate daily.

The Reality Check (Day 6): Bob runs into an unexpected issue. The version of the database he installed has a compatibility bug with the team's ORM (Object-Relational Mapping) tool. Task #3 (Data Access layer), which was supposed to take 1 day, has now consumed 3 days and still isn't finished.

In a traditional waterfall project, this delay would cause panic and push the deadline. **In XP, the system adapts:**

1. Bob raises the issue in the daily stand-up.
2. The team realizes they are going to run out of time to finish the core "Post Questions" story.
3. Because they planned a **Slack Task** (the XSS research), the team collectively agrees to **drop the slack task** for this iteration.
4. Alice, having finished her frontend design early, pairs with Bob to help him debug the database issue.

By sacrificing the low-priority slack task, the team protects the high-priority user story. The iteration stays on track.

Step 7: Iteration Review and Validation

It is Friday of Week 2, the final day of Iteration 1. The team has finished the 4 stories. They hold an **Iteration Review** meeting with the Customer Representative.

The developers demonstrate the working software. The Customer logs in, navigates the Opening Screen, and clicks "Post Question." They type a question and paste a block of Python code. The system successfully saves the question, and the code block is rendered in a beautiful, differentiated, color-coded layout.

The Customer Representative verifies that the implementation matches the verbal specifications and formally **accepts** the story. The iteration is a success.

Step 8: Moving Forward

The team holds a brief retrospective to discuss what went well and what didn't (e.g., "We need to check database version compatibility before starting a task next time").

On Monday, the cycle begins again. The team looks at Iteration 2 (Gamify, Search, Tags, Comments), breaks them down into tasks, adds new slack tasks, and begins coding.

6.4.5 Frequently Asked Questions

How long should an iteration be?

There isn't a definitive answer, as numerous factors such as team characteristics, system domain, and requirements all play a part. Short iterations—typically a week long—offer quick feedback. However, they also require more customer involvement, as new software increments need weekly validation. Moreover, stories must be simple and straightforward to fit into a 1-week iteration. Conversely, longer iterations—such as a month—grant the team more time to plan and execute tasks with reduced pressure. The trade-off is delays in receiving customer feedback, which is particularly relevant when the requirements are unclear. As a result, a 2-3 week timeframe tends to provide a balance between rapid feedback and less pressure on the team. Another useful recommendation is to experiment, testing and evaluating different durations before settling on one.

What role does the customer representative play during iterations?

At the beginning of a release, the customer representative drafts the stories to be included in that release. They then validate and approve the implementation of these stories at the end of each iteration. The representative also needs to be readily available during iterations to address questions the team may have. Since user stories contain only a few sentences, such questions are likely to arise.

How to define the team's velocity?

There is no silver bullet for this issue. The definition depends on the team's experience. If the team members have already participated in projects similar to the one they are starting, this is a less difficult task. Otherwise, the team needs to test and calibrate their velocity throughout successive iterations.

What if story X is prioritized before Y, despite depending on Y?

For instance, suppose the customer representative has allocated the story Post Question to iteration 2 and Post Answer to iteration 1. Although it seems contradictory, the team should respect this order as the final decision always lies with the customer representative. But then the dilemma might be How can we post responses without having questions? To resolve this issue, placeholder questions that can't be edited by users can be used. In iteration 1, these questions will appear by default, and users will be able to respond to them.

Is integration testing a part of Extreme Programming (XP)?

In XP, testing is tiered across different levels:

- **Unit Testing (Continuous):** Developers write automated unit tests *before* writing code (Test-Driven Development). Every piece of code must pass all unit tests before it can be released.
- **Integration Testing (Frequent):** Originally, XP encouraged system-wide integration testing as a daily activity (often at the end of the day) so teams could catch incompatible interfaces early. As tooling evolved and interface stability improved, many teams shifted to continuous integration (CI) or weekly integration cycles, but testing how components interact remains central to the workflow.
- **Acceptance Testing (Iterative):** Written by the customer (or testers representing the customer) to verify that the system satisfies business requirements and user stories. These are run frequently throughout the iterations.

So while XP emphasizes *automated unit tests* run constantly in the background, integrating code and testing how modules work together is still a critical pillar of keeping the shared codebase healthy and functional.

6.5 Scrum

Scrum is an agile, iterative, and incremental framework for project management. It was first proposed by Jeff Sutherland and Ken Schwaber in 1995. Among various agile methods, Scrum is arguably the most popular and broadly applied one.

Why Scrum is Iterative and Incremental

1. Iterative (Repetition & Refinement)

Being **iterative** means developing through **repeated cycles (iterations or sprints)** where you revisit, refine, and improve what you have already built.

- **The Core Idea:** You do not try to get everything right on the first try. Instead, you build a rough version, test it, get feedback, and continuously refine it.
- **Example:** Writing an essay or a chapter. You write a rough draft (iteration 1), review and rewrite parts of it (iteration 2), and polish the grammar (iteration 3). Each cycle improves the overall quality of the same product.

2. Incremental (Building in Pieces/Steps)

Being **incremental** means adding **new features or pieces** step-by-step until the product is fully completed.

- **The Core Idea:** The final product is broken down into smaller, independent chunks or features. You deliver these pieces sequentially, with each piece adding new functionality to the system.
- **Example:** Building a house. You build the foundation first, then the framing, then the walls, then the roof, and finally the interior finish. Each step adds a tangible, functional layer to the overall structure.

How They Work Together in Scrum

When we say Scrum is both iterative and incremental, it means:

1. **Incrementally:** The team builds the product **piece by piece** (e.g., implementing user story A in Sprint 1, user story B in Sprint 2). At the end of every sprint, you have a usable, potentially shippable product increment.
2. **Iteratively:** As you build new increments, you constantly **adapt and refine** based on customer feedback and testing. If a feature built in Sprint 1 needs to change based on new insights in Sprint 3, you iterate on it.

> ⓘ **XP VS Scrum**
>
> - **Extreme Programming (XP)** is tailor-made specifically for **software development projects**, which is why it prescribes technical and engineering rules (like test-driven development, pair programming, and continuous integration).
> - **Scrum** is a broader, domain-agnostic framework for project management. Because it focuses entirely on team roles, events, and artifacts rather than code, it can be applied to software engineering just as easily as it can be used for non-software projects—such as writing a textbook!

6.5.1 Three Pillars of Scrum

In Scrum theory, the foundational framework relies on the **Three Pillars of Scrum**, which uphold every aspect of the process:

- **Transparency:** Significant aspects of the process must be visible to those responsible for the outcome, requiring a common standard and shared language across the team.(ضمان أن تكون الجوانب الرئيسية للعمل واضحة ومرئية للجميع (سواء لفريق العمل أو أصحاب المصلحة) لتوحيد الرؤية وتجنب سوء الفهم.)
- **Inspection:** Scrum artifacts and progress toward agreed goals must be inspected frequently and diligently to detect undesirable variances or problems.(مراجعة التقدم المحرز في العمل والنتائج بانتظام لاكتشاف أي انحرافات أو مشكلات مبكراً.)
- **Adaptation:** If an inspection reveals that one or more aspects of a process deviate outside acceptable limits, the process or the material being produced must be adjusted promptly to minimize further deviation.(تعديل العمليات، أو الأولويات، أو الخطط فوراً بناءً على نتائج الفحص وما استجد من متطلبات أو متغيرات من العميل.)

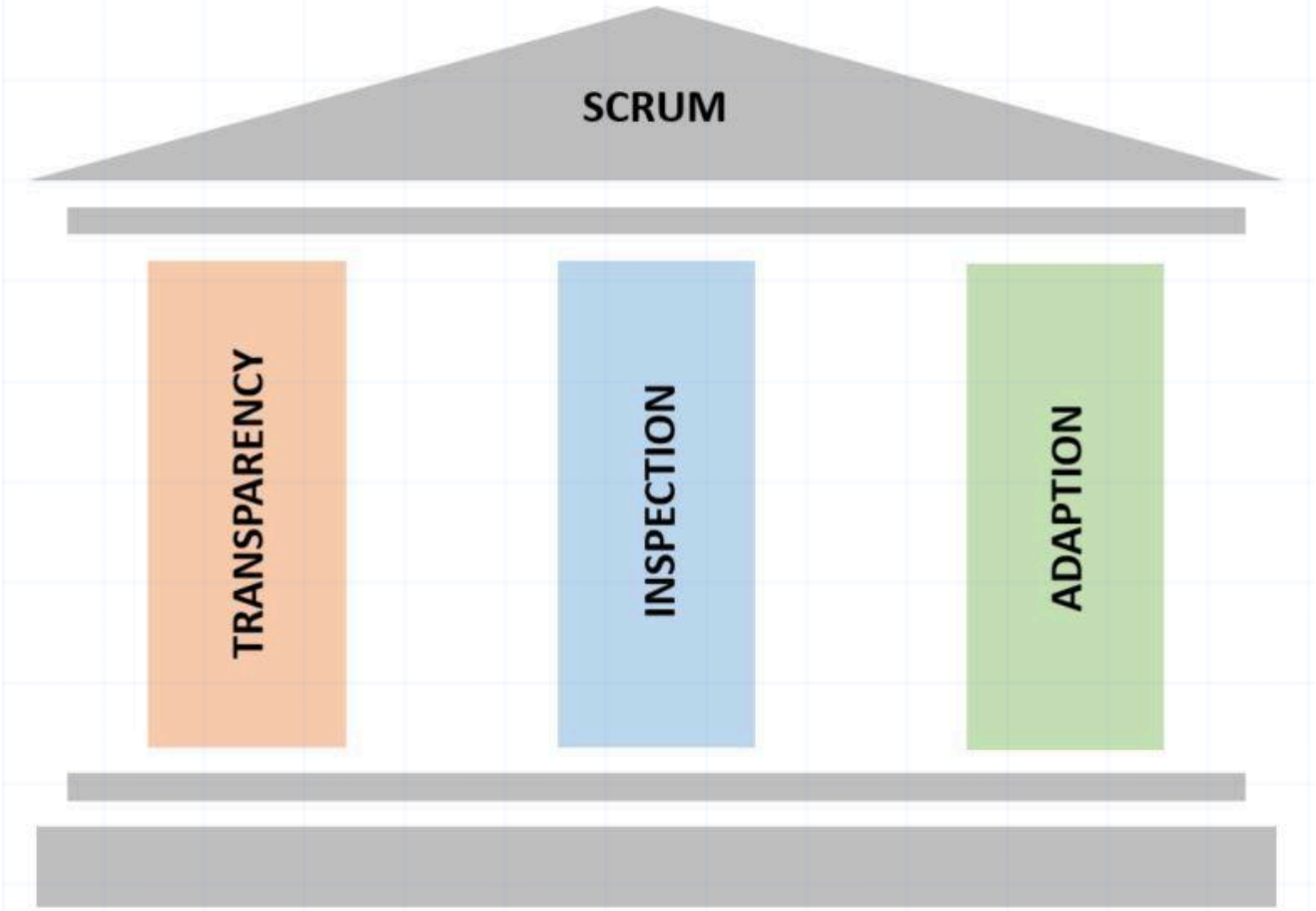


Figure 8: The Three Pillars of Scrum

6.5.2 The Core Elements of Scrum

Scrum stands out as one of the most structured agile methods because it is formally organized around these three core categories:

1. **Roles:** Product Owner (PO), Scrum Master, and Developers.
2. **Artifacts:** Product Backlog, Sprint Backlog, Scrum Board, and Burndown Chart.
3. **Events:** Sprint Planning, Sprint, Daily Meetings, Sprint Review, and Retrospective.

6.5.2.1 Roles

The Scrum Team "consists of one Scrum Master, one Product Owner, and Developers."

- Scrum Master: "accountable for establishing Scrum as defined in the Scrum Guide.". The role of the **Scrum Master** is unique to the Scrum method. They serve as the team's Scrum expert, responsible for ensuring adherence to the Scrum framework. The Scrum Master continuously trains and educates the team members on Scrum principles and acts as a facilitator to remove obstacles.
- Product Owner: "accountable for maximizing the value of the product resulting from the work of the Scrum Team.". The **Product Owner (PO)** is equivalent to the customer representative in XP. This role

requires a clear vision for the product being developed, with the aim of maximizing the return on investment for the project. The Product Owner's responsibilities include writing user stories, and they should consistently be available to address the team's questions.

- Developers: "people in the Scrum Team that are committed to creating any aspect of a usable Increment each Sprint.". Scrum teams are typically described as cross-functional or multidisciplinary, meaning they should have all necessary specialists for developing the product, thereby avoiding dependencies on external members.

The Scrum Master's focus is **process**, the Product Owner's focus is the **product (software)**, and the Developers' focus is creating a product while following Scrum processes.

6.5.2.2 The Scrum Workflow: From Artifacts to Events

Scrum manages projects by combining dynamic tracking lists (**Artifacts**) with a structured heartbeat of meetings (**Events**). Here is how they work together sequentially:

1. Defining the Vision: The Product Backlog

- **What it is:** The Product Backlog is a prioritized, dynamic list of user stories and work items proposed by the Product Owner.
- **The Workflow Role:** This is where everything starts. The Product Owner maintains and continuously updates this list as new requirements emerge or priorities change.

2. Launching the Iteration: Sprint Planning & The Sprint

- **The Sprint:** A fixed-length time-box of one month or less (often 2 weeks) that serves as the heartbeat of Scrum.
- **Sprint Planning:** This event initiates the Sprint and consists of two main parts:
    1. *Part 1 (Selecting Stories):* The Product Owner suggests top-priority stories from the **Product Backlog**, and the team evaluates if they have enough capacity/velocity to implement them.
    2. *Part 2 (Breaking Down Tasks):* Led by the developers, the selected user stories are decomposed into finer-grained, actionable tasks.
- **The Sprint Backlog:** As a direct result of Sprint Planning, the team generates the **Sprint Backlog**—a detailed list of tasks for the upcoming sprint, complete with estimated durations.

3. Tracking Daily Progress: The Scrum Board & Daily Scrum

- **The Scrum Board:** To visualize the work, teams use a board with columns (such as *To Do*,*Doing*,*Testing*, and *Done*) to track tasks and stories throughout the sprint.

- **Daily Scrum (Stand-Up):** A daily 15-minute event where developers align on progress toward the sprint goal, answer what was done yesterday and what is planned today, and surface any obstacles.

4. Closing the Iteration: Sprint Review & Retrospective

- **Sprint Review:** Held at the end of the sprint, the team demonstrates the finished product increment to key stakeholders to inspect results and gather feedback. Successfully completed stories are approved; unfinished or flawed ones return to the Product Backlog.
- **Sprint Retrospective:** A final internal meeting where the Scrum Team reflects on the past sprint to identify ways to improve team dynamics, processes, and tools for the next cycle.

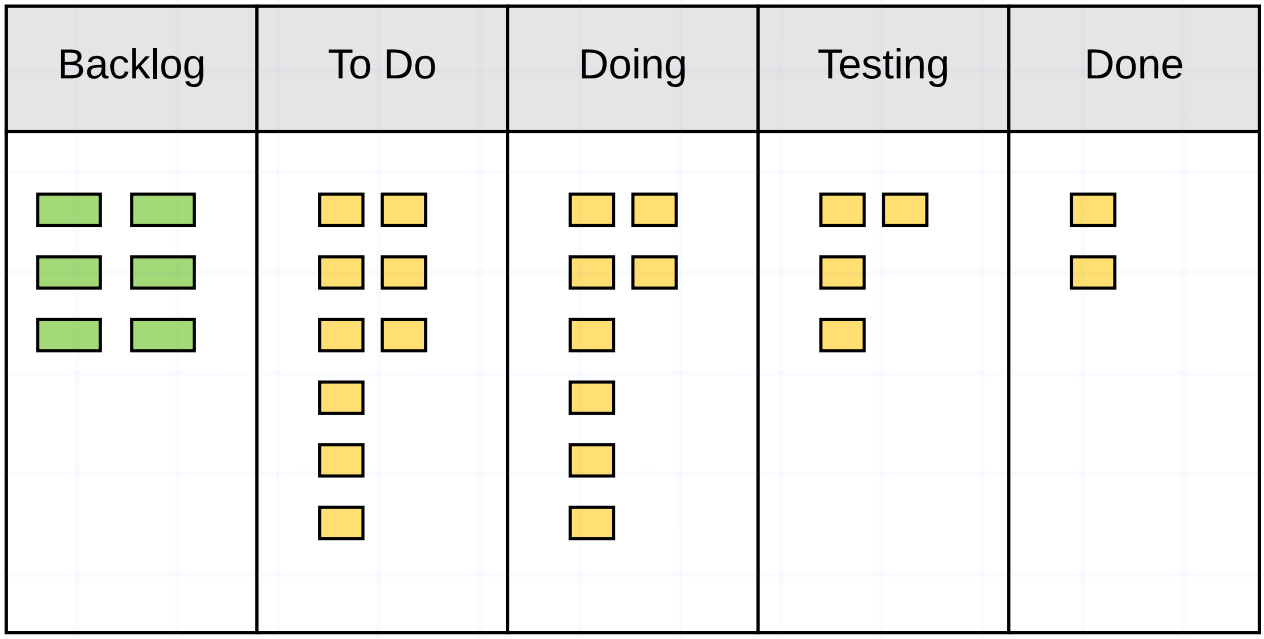


Figure 9: Example of a Scrum Board, illustrating the stories of the sprint and their corresponding tasks. Each task can fall into one of the following stages/columns: to do, doing, in testing, or done.

6.6 Kanban

The Japanese term *kanban* translates literally to visual card. Since the 1950s, the term has also been associated with the just-in-time production process used in Japanese factories, particularly at Toyota where it was first implemented. In industrial contexts, this process is also known as the Toyota Production System (TPS) or, in more contemporary terms, lean manufacturing.

In the context of software development, Kanban was first introduced at Microsoft in 2004 by David Anderson. Anderson describes Kanban as a method that promotes a sustainable work pace for development teams by eliminating waste, delivering consistent value, and fostering a culture of continuous improvement.

To better understand Kanban, we compare it with Scrum. Primarily, Kanban is simpler than Scrum because it does not have events like sprints. Furthermore, strictly speaking, Kanban does not define

specific roles like Product Owner or Scrum Master. Lastly, unlike Scrum, Kanban has fewer artifacts, focusing primarily on the task board. In Kanban, this artifact is referred to as the **Kanban Board**, which also includes the product backlog.

6.6.1 Key characteristics of Kanban

Key characteristics of Kanban include:

- **The Kanban Board:** Acts as the primary artifact, containing a backlog column followed by steps required to turn stories into functioning features (such as Specification, Implementation, and Code Review).
- **Pull System:** Tasks progress from left to right, and team members pull completed items into the next step when capacity allows.
- **Work in Progress (WIP) Limits:** Enforces maximum task limits for each step on the board to prevent teams from becoming overburdened and to maintain high quality (calculated using **Little's Law**, where $WIP = TP \times LT$).

6.6.2 Kanban Board

The Kanban Board is organized into specific columns, as described below:

- The first column represents the product backlog, closely mirroring the same artifact in Scrum. User stories are proposed and subsequently added to this backlog.
- The next columns represent the steps required to transition a user story into a functioning feature. These columns are typically labelled as Specification, Implementation, and Code Review. The stories from the backlog are processed sequentially, mirroring an assembly line workflow that progresses from left to right. Each column is further sectioned into two subcolumns labeled *in progress* and *ready*. For instance, the Implementation column contains two subcolumns: tasks under implementation and tasks that have been implemented. Completed tasks from one step await being pulled into the next step by a team member. This approach is why Kanban is often called a pull system.

📝 **EXAMPLE 2:**

Backlog	Specification WIP		Implementation WIP		Code Review WIP	
	in progress	ready	in progress	ready	in progress	done
H3	**H2**	**T6 T7 T8 T9**	**T4 T5**	**T3**	**T2**	**T1**

In this board, we can see stories and tasks at various stages. In the backlog, we have one story labeled H3. Another story, labeled as H2, has been picked up by a team member and is under Specification. Four tasks from a previous story, labeled as T6 to T9, have already been specified. Next, there are two tasks, T4 and T5, which are being implemented. Task T3 has been implemented and is now awaiting Code Review. Lastly, Task T2 is currently under review, while T1 has been fully implemented and is now considered done. You may also have noticed the acronym WIP appearing at all steps, with the exception of the backlog. Don't worry about this for now; we will explain it shortly. It's worth noting that, in this example, we used the letters H and T to represent stories and tasks, respectively. However, in real-world Kanban boards, tasks and stories are represented as cards that carry a brief description of each item.

Days later, the Kanban Board was updated to the following state (tasks that moved forward are distinguished by being underlined and in red).

Backlog	Specification WIP		Implementation WIP		Code Review WIP	
	in progress	ready	in progress	ready	in progress	done
H3		T8 T9 T10 T11 T12	T4 T5 T6 T7		T3	T1 T2

Note that story H2 is no longer visible, as it was decomposed into three tasks (T10, T11, and T12). This decomposition is precisely what occurs during the specification phase, whose objective is to convert a story into an actionable task list. Furthermore, T6 and T7—which were previously on hold—are now under Implementation. Meanwhile, T3 has transitioned into Code Review. Finally, the review of T2 has been completed. It's also important to note that currently there are no tasks waiting to enter the review stage.

6.6.2.1 Work in Progress Limits

WIP Limits (short for **Work in Progress Limits**) represent the maximum number of tasks or cards that can be present in each step of a Kanban board at any given time. Kanban uses WIP limits to enforce a sustainable work pace and prevent two major issues:

1. **Teams having no tasks to work on** and staying idle.
2. **Teams being overwhelmed with an excess of tasks**, which creates bottlenecks and negatively impacts software quality.

By placing a strict cap on columns like *Specification*,*Implementation*, or *Code Review*, WIP limits act as an automatic "lock" that prevents the team from accepting more work than their actual delivery capacity. This helps teams finish existing tasks before starting new ones.

Chapter 3 Requirements Engineering

> *The hardest single part of building a software system is deciding precisely what to build*
>
> — Frederick Brooks

1 Introduction

Before we start to develop our software, it becomes quite essential for us to understand and document the exact requirement of the customer. Experienced members of the development team carry out this job. They are called as **system analysts**.

> 📖 DEFINITION 1: **Requirement**
>
> A requirement is a feature of the system or a description of something the system is capable of doing in order to fulfill the system's purpose.

2 Core Questionnaire

During the requirements gathering and analysis phase, a system analyst must bridge the **gap between the customer's vision and the technical specifications needed for development.** To gain a thorough and unambiguous understanding of the project, the analyst should systematically address the following foundational questions:

1. Problem Definition & Justification

- **What is the problem?**

  Define the exact pain points, inefficiencies, or operational gaps in the current workflow that necessitate the development of the software.

- **Why is it important to solve the problem?**

  Establish the business value, urgency, and expected impact of addressing these issues on the organization or end-users.

2. Solution Exploration & Scope

- **What are the possible solutions to the problem?**

  Explore alternative approaches (e.g., custom development vs. commercial off-the-shelf software, or varying architectural paradigms) to ensure the most viable path is selected during the feasibility study.

3. Data Flow & Boundaries

- **What exactly are the data input to the system and what exactly are the data output by the system?**

  Identify the precise data sources, formats, and domains entering the system, as well as the expected outputs, reports, or results generated for the user. (This lays the groundwork for creating Data Flow Diagrams and Data Dictionaries).

4. Technical Complexities & Constraints

- **What are the likely complexities that might arise while solving the problem?**

  Pinpoint technical, architectural, or domain-specific risks (such as performance bottlenecks, concurrency hurdles, or algorithmic complexities) that need mitigation.

5. System Interfacing & Integration

- **If there are external software or hardware with which the developed software has to interface, then what exactly would the data interchange formats with the external system be?**

  Specify the communication protocols, APIs, file structures (e.g., JSON, XML, binary formats), or hardware constraints required for seamless third-party integration.

6. Stakeholders & Business Value

- **Who is requesting the software?**

  Identify the sponsor or business stakeholder funding/initiating the project and defining its high-level objectives.

- **Who will use the software?**

  Characterize the primary and secondary end-users, distinguishing between different user personas and their distinct interaction requirements.

- **What is the benefit that the software will bring?**

  Summarize the qualitative and quantitative returns (e.g., time saved, error reduction, increased throughput, cost efficiency) delivered upon successful deployment.

Note

After the analyst has understood the exact customer requirements, he proceeds to identify and resolve the various requirements problems. The most important requirements problems that the analyst has to identify and eliminate are the problems of anomalies, inconsistencies, and incompleteness. When the analyst detects any inconsistencies, anomalies or incompleteness in the gathered requirements, he resolves them by carrying out further discussions with the end users and the customers.

3 What is requirements engineering?

📖 DEFINITION 2: **Requirements Engineering**

The process concerned with understanding the software system that the customer wants developed (and not how it should be developed).

3.1 Lifecycle of Requirements Engineering

Requirements engineering consists of the following processes where Each process builds systematically on the previous one to ensure that what gets built actually solves the right problem for the customer:

- **Inception:** Establishing the foundational understanding and setting the boundaries of what the system will touch in which the nature and scope of the system is defined.
- **Elicitation:** Interacting directly with stakeholders, users, and domain experts to pull out what they need the system to do.
- **Elaboration:** Taking those raw, often vague requirements and breaking them down into more detailed technical and operational models.
- **Negotiation:** Balancing competing stakeholder interests, trimming unrealistic feature lists, and resolving conflicting demands within budget and time constraints.
- **Specification:** Capturing the refined agreements in a formal artifact—typically a Software Requirements Specification (SRS) document or a structured model.
- **Validation:** Reviewing and auditing the documented requirements to guarantee they are accurate, feasible, unambiguous, and complete before heavy design begins.
- **Management:** Controlling how requirements evolve, handling change requests, and maintaining traceability as the project progresses through its lifecycle.

> **Note**
>
> These steps are overlapping for a variety of reasons. You should be able to notice that some of them, such as management, must occur throughout the communication and modelling activities. Negotiation will also occur at each of the various requirements engineering steps. More importantly, a process model which understands that requirements may be (initially) poorly understood, and that they may change through the project's lifetime, will also iteratively collect and detail the use cases (consider the unified process model from the previous chapter). This iterative process will forcefully overlap all of these steps

4 Types of software requirements

Software requirements are often categorized into these overlapping dimensions:

1. **By Nature:** Functional vs. Non-Functional.
2. **By Viewpoint / Abstraction:** User Requirements (what the user sees/wants) vs. System Requirements (how the system is specified technically).
3. **By Lifecycle Priority:** Mandatory/Essential (must-have) vs. Desirable (nice-to-have).

4.1 Functional versus Non-Functional Requirements

4.1.1 Functional requirements (FRs)

Functional requirements (FRs) define the fundamental behaviors, features, and business logic of a software system. They explicitly state what the system *should do*, how it must react to specific inputs, and how it should behave under predictable operational scenarios. In essence, functional requirements map directly to the transformations applied to input data to generate the expected output data structures:

$$\text{Inputs} \xrightarrow{\text{Functional Logic}} \text{Processed Data/Outputs}$$

If a system fails to fulfill a functional requirement, the product is fundamentally broken or incomplete with respect to its core purpose.

> **EXAMPLE 1: Online Banking System**
>
> Consider an online banking platform. The functional requirements dictate the core transactional capabilities available to both customer and administrative actors:
>
> 1. **Account Balance Reporting:** The system must retrieve and display the current available balance for a user's checking account within 1.5 seconds of a successful authentication

request.

2. **Fund Transfers:** The system must allow authenticated users to transfer funds between internal accounts or execute wire transfers to external bank accounts, provided the source account contains sufficient funds.
3. **Debit Card Management:** The system must provide a dedicated interface allowing users to temporarily lock or permanently cancel their debit cards.

4.1.1.1 Characteristics of Well-Formed Functional Requirements

- **Determinism:** Given a specific input state S_0 and a trigger event E, the system must transition to a predictable output state S_1.
- **Completeness:** Every user interaction, data processing rule, and system calculation must be accounted for without ambiguous edge cases.
- **Traceability:** Functional requirements must link directly to high-level user stories, use case models, and subsequent system-level design artifacts (such as class diagrams or data-flow diagrams).

4.1.2 Non-Functional Requirements

Non-functional requirements (NFRs)—frequently referred to as **quality attributes**,**constraints**, or **system qualities**—define *how well* the system performs its functions. Rather than describing specific features, NFRs establish the operational, environmental, and architectural boundaries under which the system must operate.

While a functional requirement dictates that the system *must* process a bank transfer, a non-functional requirement dictates that the transfer interface *must* maintain 99.99% availability and encrypt all transaction payloads using AES-256 standards.

> ⓘ **Note**
>
> Functional requirements are frequently specified in **natural language** (e.g., in English). Conversely, non-functional requirements are specified using **metrics.**

4.1.2.1 Quantifying Non-Functional Requirements

Because non-functional requirements dictate *how well* a system operates rather than what features it provides, they are notoriously susceptible to vague and ambiguous phrasing. Statements like "*the system will have high availability*" or "*the response time will be quick*" are untestable because "high" and "quick" mean different things to different stakeholders.

To prevent ambiguity during design, implementation, and acceptance testing, professional software specifications mandate that **non-functional requirements must be defined using objective metrics**.

Table 2: *Standard Non-Functional Metrics Matrix*

Non-Functional Requirement	Description & Focus	Standard Engineering Metric	Example Quantification
Performance	Speed, latency, throughput, and resource responsiveness.	Transactions per second (TPS), response time, latency, throughput.	99% of transactions executed in any 5-minute window must have a response time $\leq 1.0\ \text{second}$.
Space / Storage	Efficiency in utilizing hardware storage and memory capacity.	Storage usage, RAM consumption, cache footprint.	The mobile client app package size must not exceed $50\ \text{MB}$, and memory usage during idle state must remain under $100\ \text{MB}$.
Reliability	System dependability and continuity of service.	Uptime percentage, Mean Time Between Failures (MTBF).	System uptime of 99.99% (allowing a maximum cumulative downtime of $\approx 52.6\ \text{minutes/year}$).
Robustness / Fault Tolerance	Ability to recover from unexpected failures or erroneous inputs.	Mean Time To Recover (MTTR), risk of data loss after crash.	System must automatically recover from database node failures within $30\ \text{seconds}$ with zero data loss.
Usability	Ease of learning, efficiency, and user satisfaction.	User training time, task completion rate, error frequency.	A novice user must be able to complete checkout within $3\ \text{minutes}$ after a maximum of $15\ \text{minutes}$ of training.
Portability	Ease of adapting software across different platforms or environments.	Percentage of portable lines of code, environment dependencies.	100% of core business logic must be platform-agnostic and compile natively on Linux, macOS, and Windows.

4.2 User Requirements versus System Requirements

Beyond classifying requirements by their intrinsic nature (functional vs. non-functional), requirements engineering categorizes specifications by **viewpoint and abstraction**. Originally popularized in software engineering literature by Ian Sommerville, this perspective differentiates between the high-level needs of the domain stakeholders and the rigorous technical specifications required by developers and architects.

4.2.1 User Requirements

User requirements are high-level statements written in natural language, accompanied by intuitive diagrams or operational models, that describe what the user expects the system to accomplish. They focus entirely on the **problem domain** rather than the technological solution.

User requirements are typically authored by business analysts, product owners, or end-users themselves. They should be accessible to non-technical stakeholders, avoiding internal system architecture jargon, database schemas, or low-level interface protocols.

Characteristics & Best Practices

- **Audience:** Designed for clients, managers, domain experts, and end-users.
- **Language:** Natural language combined with sketches, use case summaries, or user stories.
- **Focus:** States *what* services the system is expected to provide and under what operational constraints the business operates, without prescribing how the software will implement them internally.

4.2.2 System Requirements

System requirements—often referred to in engineering specifications as the **Functional System Specification (FSS)**—are detailed, precise, and structured descriptions of the software's behavior, internal data structures, and operational constraints. They bridge the gap between the business problem and the software solution, serving as the definitive contract between the customer and the development team.

While a user requirement might state that a user needs to securely transfer funds, the corresponding system requirements specify the exact cryptographic protocols, API endpoints, database transaction isolation levels, and error-handling routines required to make that transfer happen.

Characteristics & Best Practices

- **Audience:** Designed for software architects, developers, testers, and system administrators.
- **Language:** Structured natural language, formal specification languages, sequence diagrams, class diagrams, and state transition models.
- **Focus:** States *how* the system will be built, detailing technical components, interfaces, algorithms, and compliance standards.

4.2.3 The Expansion Relationship: From User to System

A fundamental principle of requirements engineering is that **a single user requirement frequently expands into a set of multiple system requirements**.

To illustrate this expansion, consider the following mapping from an online banking domain:

Requirement Level	Specification Statement	Description & Focus
User Requirement (UR_1)	*"The system must allow registered customers to transfer funds securely from their checking account to an external bank account via wire transfer."*	**Problem-Oriented:** Written in natural language for the bank customer and business analyst. Focuses on business utility and user goals.
System Requirement ($SR_{1.1}$)	*"The transfer module must validate that the source account balance $\geq$ requested transfer amount plus applicable wire fees prior to initiating the transaction."*	**Solution-Oriented:** Specifies internal business logic constraints and validation rules.
System Requirement ($SR_{1.2}$)	*"All external wire transactions must communicate with the SWIFT gateway using TLS 1.3 encryption and mutual certificate authentication."*	**Technical Constraint:** Specifies security protocols, encryption standards, and interface communication channels.
System Requirement ($SR_{1.3}$)	*"If an external banking gateway timeout (≥ 5.0 seconds) occurs, the transaction must roll back atomically and display error code `ERR_GW_TIMEOUT` to the user."*	**Fault Tolerance:** Specifies technical error handling, state rollback rules, and exception codes for developers and testers.

4.3 Examples

EXAMPLE 2:

Draft Requirement: *"The portal system should be fast, secure, and let students register for courses while keeping all student data accurate and consistent across the campus network."*

Show Solution

Is it Functional or Non-Functional?

- It is a **hybrid (mixed) requirement**.

- "*let students register for courses*" is a **Functional Requirement (FR)** because it describes a specific system feature/behavior.
- "*fast, secure, and keeping all student data accurate and consistent*" describes performance, security, and data integrity attributes, which are **Non-Functional Requirements (NFRs)**.

Is it complete and correct?

- **No**, it is incomplete and poorly structured.
- **Missing Details:** It fails to specify *which* courses can be registered, what validation rules apply (e.g., prerequisite checks), what security standard or encryption protocol is required, and what constitutes "fast" response times.

Can a QA engineer test this as written?

No. Words like "fast" and "secure" are subjective and qualitative . A testing team cannot write an objective pass/fail test for whether a system is "fast" unless concrete metrics (like milliseconds of latency or transactions per second) are provided .

Refinement & Rewriting

To make this professional, precise, and **verifiable**, the single compound sentence must be split into distinct, measurable requirements:

1. **Functional Requirement (FR):**
   - "*The portal system shall allow authenticated students to add and drop courses during official registration windows, validating that prerequisite courses have been successfully completed prior to enrollment.*"
2. **Non-Functional Requirement - Performance (NFR):**
   - "*The portal system shall ensure that* 99% *of course registration requests are processed with a maximum response time of* ≤ 1.5 seconds *during peak load.*"
3. **Non-Functional Requirement - Security & Privacy (NFR):**
   - "*All student data transmitted across the campus network must be encrypted using TLS 1.3, and passwords must be hashed using bcrypt.*"

📝 **EXAMPLE 3:**

Imagine your team is developing the **AAUP Digital Course Evaluation and Feedback Module**. A junior developer wrote the following non-functional requirement for the system:

> **Draft Requirement:** *"The student course evaluation module must be extremely reliable and handle heavy traffic whenever final semester grades are published."*

- **Identify the Flaw:** Why is this requirement impossible to test or verify objectively by a Quality Assurance (QA) engineer?
- **Your Turn:** How would you rewrite and quantify this requirement into a **verifiable** non-functional requirement (using proper metrics like uptime, availability, or response times) so that it can be objectively tested?

✅ Show Solution

This requirement is **completely qualitative, vague, and untestable**.

- Words like **"extremely reliable"** and **"handle heavy traffic"** are subjective.
- A Quality Assurance (QA) engineer cannot write a test case for "extremely reliable" because there is no numerical threshold. If the system crashes under 10,000 users, does that count as handling heavy traffic? Without explicit metrics, nobody can prove whether the requirement has passed or failed.

2. How to Rewrite and Quantify It (To Make it Verifiable)

To make this non-functional requirement **verifiable**, it must be rewritten using objective metrics for **reliability** and **performance/throughput** (following standard guidelines found in software engineering textbooks like):

- **Rewritten Verifiable Requirement - Reliability:**

  > *"The student course evaluation module shall maintain an availability (uptime) of* 99.9% *during grade release week, with a Mean Time To Recover (MTTR) of less than* 15 minutes *in the event of any unexpected failure."*

- **Rewritten Verifiable Requirement - Performance & Capacity:**

  > *"The module shall support a minimum throughput of* 500 $\text{transactions per second (TPS)}$ *and maintain a maximum response time of* ≤ 1.5 seconds *during peak final grade publishing periods."*

5 The Requirements Specification Document (RSD)

Once requirements are elicited from stakeholders, they undergo a rigorous process of analysis, refinement, and organization. In traditional or plan-driven software engineering projects (such as those following waterfall-based lifecycles), these requirements are formalized into a comprehensive artifact known as the **Requirements Specification Document (RSD)** or **Software Requirements Specification (SRS)**.

The RSD serves as the definitive technical blueprint and contract between the customer, project managers, and the development team, establishing precisely what the software must accomplish prior to architectural design and coding.

> **Agile Methodologies**
>
> In Agile development, requirements are documented using **user stories** rather than relying on massive upfront requirement documents, agile frameworks favor lightweight, iterative backlogs that evolve continuously in close collaboration with the customer. However, in plan-driven projects, enterprise contracts, or safety-critical domains, a formal Requirements Specification Document remains necessary.

5.1 IEEE requirements standard

To bring discipline and uniformity to software documentation in traditional models, the **IEEE 830 Standard** was introduced in the 1990s. Although agile teams favor flexibility, the structural layout established by IEEE 830 provides a robust template for large-scale systems where comprehensive documentation is contractual and mandatory.

An IEEE 830-compliant document systematically organizes requirements into clearly defined sections to eliminate ambiguities.

IEEE Guide to Software Requirements Specifications — OPEN PDF

IEEE Recommended Practice for Software Requirements Specifications
https://standards.ieee.org/ieee/830/1222/ — OPEN LINK

5.1.1 IEEE 830 Standard SRS Structure & Main Headings

- **Software Requirements Specification (IEEE 830)**
  - **1. Introduction**
    - 1.1 Purpose

- 1.2 Document Conventions
- 1.3 Intended Audience and Reading Suggestions
- 1.4 Product Scope
- 1.5 References
- **2. Overall Description**
  - 2.1 Product Perspective
  - 2.2 Product Functions
  - 2.3 User Classes and Characteristics
  - 2.4 Operating Environment
  - 2.5 Design and Implementation Constraints
  - 2.6 Assumptions and Dependencies
- **3. Specific Requirements (The Core Technical Section)**
  - **3.1 External Interface Requirements**
    - 3.1.1 User Interfaces
    - 3.1.2 Hardware Interfaces
    - 3.1.3 Software Interfaces
    - 3.1.4 Communications Interfaces
  - 3.2 Functional Requirements
  - 3.3 Performance Requirements
  - 3.4 Logical Database Requirements
  - 3.5 Design Constraints
  - 3.6 Software System Attributes (Reliability, Availability, Security)
  - 3.7 Other Requirements
- Appendices
- Index

Detailed Description of Main Sections

1. Introduction

- **Purpose:** Defines the goals and objectives of the software product being specified.
- **Document Conventions:** Explains any formatting standards, typography, or notation rules utilized throughout the document.
- **Intended Audience:** Identifies who will read the document (e.g., developers, project managers, clients, testers) and suggests reading paths.
- **Product Scope:** Outlines what the software will and will not do, linking it to broader organizational or business goals.

- **References:** Lists related documents, standards, or source materials.

2. Overall Description

- **Product Perspective:** Contextualizes the software within its environment, showing whether it is a standalone product, a component of a larger system, or a replacement for legacy software.
- **Product Functions:** Summarizes the major capabilities and high-level features the software will execute.
- **User Classes and Characteristics:** Identifies the different types of users (e.g., administrators, regular users, guests) and details their technical expertise and frequency of use.
- **Operating Environment:** Describes the hardware, operating system, and network infrastructure where the software will run.
- **Constraints & Assumptions:** Outlines any regulatory limitations, hardware restrictions, or underlying assumptions that could affect the project.

3. Specific Requirements

This is the largest, most rigorous section containing all the detailed technical specifications required by developers and testers:

- **External Interface Requirements:** Detailed specifications for how the system connects with humans (UI), hardware devices, other software, and communication protocols.
- **Functional Requirements:** The core behavioral features, input/output processing rules, and computational logic of the system.
- **Performance Requirements:** Quantified metrics regarding speed, throughput, response time, and capacity.
- **Database Requirements:** Data structures, logical storage schemas, and retention policies.
- **System Attributes (Non-Functional):** Security, privacy, maintainability, availability, and reliability metrics.

Appendices & Index

- Includes supplementary material such as analysis models, UI wireframes, rough cost estimations, or glossaries of domain-specific terminology.

5.2 AAUP Integrated Student & Campus Portal Module

5.2.1 Purpose & Scope

- **Purpose:** To specify the complete functional requirements, user workflows, and technical performance constraints for the new **AAUP Campus Portal**, a centralized web and mobile platform designed to streamline academic registration, tuition tracking, course management, and campus services for students and faculty.
- **Scope:** The system encompasses a responsive web portal and mobile application interface, backend database integration with the university's central Student Information System (SIS), and secure APIs for external campus services (such as parking access and library checkouts).

5.2.2 Definitions, Acronyms, and Abbreviations

- **SIS:** Student Information System.
- **GPA:** Grade Point Average.
- **API:** Application Programming Interface.
- **SSO:** Single Sign-On (institutional authentication).
- **RFID:** Radio Frequency Identification.
- **MFA:** Multi-Factor Authentication.

5.2.3 Overall Description

5.2.3.1 Product Perspective & Environment

The AAUP Portal operates as a cloud-hosted, multi-tier web application. It integrates directly with existing legacy databases on campus, authenticates users via institutional Microsoft/Exchange credentials, and serves thousands of concurrent users during peak academic periods (such as course registration week). Furthermore, it interfaces with campus entry RFID gate scanners to sync student parking access and attendance logs in real time.

5.2.3.2 Product Functions (Summary of Capabilities)

- **Academic Management:** Course registration, grade reporting, and real-time degree audit tracking.
- **Financial Ledger:** Tuition fee calculations, online payment gateway integration, and financial clearance tracking.
- **Communication Hub:** Official announcements from faculty members, department heads, and administration.
- **Campus Integration Services:** Real-time parking lot availability checks and digital campus service hooks.

5.2.3.3 User Classes and Characteristics

- **Undergraduate & Graduate Students:** High-volume users accessing the portal daily for schedules, grades, and announcements. They require an intuitive, fast mobile-first interface.
- **Faculty Members & Instructors:** Academic staff responsible for posting grades, uploading course materials (lecture notes and syllabi), and managing class rosters via desktop workstations or mobile screens.
- **Administrative Staff / Admissions:** Users with elevated privileges to manage student records, resolve registration blocks, and generate institutional accreditation reports.

5.2.3.4 Operating Environment & Constraints

- The portal must be fully accessible across modern web browsers (Chrome, Firefox, Safari, Edge) and mobile operating systems (iOS and Android).
- The system must handle high traffic surges during the first week of semester add/drop periods without performance degradation.
- System maintenance windows must be restricted to designated off-peak weekend hours to avoid disrupting active academic periods.

5.2.4 Specific Requirements

5.2.4.1 External Interface Requirements

- **User Interfaces (UI):** Responsive design adapting automatically to smartphone screens, tablets, and desktop workstations used in faculty offices, adhering strictly to AAUP institutional branding guidelines.
- **Hardware Interfaces:** The backend must interface with passive RFID gate readers at campus entry points to read student/staff parking credentials and log entry timestamps.
- **Software Interfaces:** Integration with the university's Microsoft Exchange/Active Directory for Single Sign-On (SSO) authentication, the email server for automated notifications, and the accounting department's payment gateway for online tuition processing.
- **Communication Interfaces:** All data exchanges between clients, mobile applications, and cloud backend servers must communicate via HTTPS using TLS 1.3 encryption protocols.

5.2.4.2 Functional Requirements (Grouped by Feature)

- **FR-01 (Course Registration):** The system must allow authenticated students to add or drop elective and core courses during official registration windows, validating prerequisites (e.g., ensuring a student has passed *Introduction to Database Systems* before registering for advanced courses) in real-time.

- **FR-02 (Grade Posting):** Instructors must be able to input, verify, and submit final semester grades for class rosters, with the system permanently locking submissions after the official academic calendar deadline passes.
- **FR-03 (Financial Clearance Check):** The system must automatically check a student's tuition balance ledger and block course registration if outstanding dues exceed the financial threshold set by the university administration.
- **FR-04 (Parking Gate Sync):** When a registered vehicle passes an entry RFID scanner, the system must update lot availability metrics and verify active student status instantaneously.

5.2.4.3 Non-Functional Requirements (Quantified Metrics)

- **Performance:** Page load times for student grade reports and course schedules must not exceed $1.5\ \text{seconds}$ under normal operational load, with maximum query latencies under $1.0\ \text{second}$ for 99% of transactions during peak hours.
- **Availability:** The portal must maintain an uptime of 99.9% throughout the academic semester.
- **Security & Privacy:** All user sessions must be encrypted using TLS 1.3, passwords must comply with institutional security policies, and accounts must be protected via multi-factor authentication (MFA).

6 User Stories

User Stories are popular, lightweight instruments used primarily in **Agile methods** to define software requirements from an end-user's perspective.

Instead of writing massive, traditional waterfall documents that take months to complete, Agile teams use user stories to capture requirements concisely, keeping the focus on *who* needs the feature,*what* they want, and *why* they need it.

> ⓘ **Note**
>
> It is also recommended to define the key users who will interact with the system before writing stories.

6.1 The Three Cs of a User Story

As defined by Ron Jeffries, a user story consists of three vital parts:

1. **Card:** A physical or digital index card containing a short, natural-language sentence written by the customer or product owner describing a feature.

2. **Conversations:** Verbal discussions between the developers, testers, and customers to flesh out the detailed specifics, edge cases, and acceptance criteria.
3. **Confirmation:** The acceptance tests that prove the user story has been implemented successfully.

6.2 User Stories properties

- **Independent:** It should be possible to implement any two stories in any order. Ideally, there should be no dependencies between stories.
- **Negotiable:** As mentioned earlier, stories (the card) are invitations for conversations between customers and developers during a sprint. Both parties should be open to changing and adapting their opinions as a result of these discussions. Developers should be open to implementing details not expressed on or that do not fit on the story cards. Customers should also consider technical arguments from developers, such as those regarding the complexity of implementing some aspects of the story as initially planned.
- **Valuable:** Stories should add value to the customers' business. Stories are proposed, written, and ranked by the customers according to the value they add to their business. Consequently, the idea of a technical story—such as the system must be implemented in JavaScript, using React and Node.js—is inappropriate.
- **Estimable:** It should be possible to estimate the size of a story, i.e., to define the effort needed to implement it. Typically, this requires the story to be small, as we will discuss later. Estimation also becomes easier when the developers have experience in the system's domain.
- **Small:** Complex and large stories—also known as **epics**—can exist, but they should be placed at the bottom of the backlog, indicating they will not be implemented soon. On the other hand, stories at the top of the backlog should be short and small to facilitate understanding and estimation. For a one-month sprint, it should be possible to implement any story in less than one week, for example.
- **Testable:** Stories should have clear acceptance tests. For example, the customer can pay by credit card is testable, provided we know the kind of credit card that will be used. Conversely, a story like a customer should not have to wait too long to have their purchase confirmed is not testable. This is because it lacks a clear acceptance test.

6.3 Standard Format of a User Story

The most common template used to write a user story is:

> **As a** [Type of User], **I want** [some goal or feature], **So that** [some business value or benefit].

🎬 SCENARIO 1: Library Management System

In this section, we provide examples of user stories for a library management system. These stories are associated with three user types: students, instructors, and library staff.

First, we present stories suggested by students (see below). Any library user can perform the operations described in these stories. Note that the stories are brief and do not elaborate on how each operation should be implemented. For example, one of the stories states that students should be able to search for books. However, many details are omitted, including search criteria, available filters, limits on the number of search results, and the layout of search and results pages. Nonetheless, we should remember that a story is essentially a commitment: the customer representative ensures they will be available to clarify these details with developers during the sprint in which the story is implemented. When working with user stories, this verbal interaction between developers and the customer representative is crucial to successful requirements specification and implementation.

> As a student, I want to borrow books.
>
> As a student, I want to return books I have borrowed.
>
> As a student, I want to renew my book loans.
>
> As a student, I want to search for books.
>
> As a student, I want to reserve books that are currently on loan.
>
> As a student, I want to receive notifications about new acquisitions.

Now, we present the stories suggested by the instructors:

> As an instructor, I want to borrow books for an extended period of time.
>
> As an instructor, I want to recommend books for acquisition.
>
> As an instructor, I want to donate books to the library.
>
> As an instructor, I want to return books to other campus libraries.

Although these stories originate from instructors, this does not mean they are exclusive to this user group. For example, during the sprint, the customer representative (or Product Owner) may consider making the donation feature available to all users.

The last story proposed by instructors—allowing books to be returned to any university library—can be classified as an **epic**, that is, a complex story. This story refers to a scenario where an instructor borrows a book from the central library but wants to return it to a departmental library, or vice versa. Implementing this story is more complex as it requires integrating different library systems and having staff members transport the books back to their original locations.

Finally, we share the stories proposed by the library staff members, typically concerning library organization and ensuring its seamless operation:

> As a staff member, I want to register new users.
>
> As a staff member, I want to add new books to the system.
>
> As a staff member, I want to remove damaged books from the system.
>
> As a staff member, I want to access statistics about the collection.
>
> As a staff member, I want the system to send reminders to users with overdue books.
>
> As a staff member, I want the system to apply fines for late book returns.

To confirm the implementation of the search story, the customer representative defined the following test scenarios:

> Search for books using an ISBN.
>
> Search for books using an author's name.
>
> Search for books using a title.
>
> Search for books added to the library on or after a specific date.

The correct implementation of these searches will be demonstrated during the Sprint Review meeting, assuming the team is using Scrum.

As previously mentioned, acceptance tests are specified by the customer representative (or Product Owner). This practice helps prevent a scenario known as **gold plating**. In Requirements Engineering, this term describes the situation where developers independently decide to elaborate on certain stories—or requirements, more generally—without the customer's input. Metaphorically, developers are embellishing stories with layers of gold, which does not generate value for users.

6.4 The Lifecycle of User Stories in Agile

- **Created & Added Directly to the Product Backlog:**
  - When user stories (such as *"As a student, I want to borrow books"*) are first elicited and written during workshops or brainstorming sessions,**they are added directly to the Product Backlog** (not a sprint log).
  - The Product Backlog acts as the master, prioritized inventory of all desired features, enhancements, and requirements for the product.
- **Managed & Prioritized by the Product Owner (PO):**
  - The **Product Owner** owns, maintains, and continuously prioritizes the Product Backlog based on business value, user needs, and stakeholder feedback. High-value stories are placed at the top, while complex or low-priority stories (like *epics*) stay at the bottom.
- **Refinement (Backlog Grooming):**
  - Before stories are pulled into an upcoming development cycle, the team goes through **Backlog Refinement**. During these sessions, developers, testers, and the Product Owner discuss the stories to:
    - Clarify ambiguities.
    - Break down large items (*epics*) into smaller, independent stories.
    - Estimate the effort required (sizing/story points).
    - Define the **Acceptance Tests** (Confirmation) so everyone agrees on what "done" means.
- **Moved to the Sprint Backlog (Sprint Log):**
  - When a new sprint planning meeting begins, top-priority, well-refined stories are pulled from the *Product Backlog* and moved into the **Sprint Backlog** (Sprint Log) for the upcoming iteration (e.g., a 2-week sprint). Only then are they actively worked on, designed, coded, and tested by the development team.

📝 EXAMPLE 4: **Case Study: Backlog Grooming and Task Breakdown for Book Search**

Phase 1: Initial Elicitation & Adding to the Product Backlog

> **User Story Card (Draft):**
>
> - **As a** student,

> - **I want** to search for books,
> - **So that** I can quickly find the materials I need for my courses.

- **Where it goes:** This rough, high-level story is added directly to the **Product Backlog** as a prioritized item, but it is currently too broad (an *epic* or unrefined story) to be coded immediately.

Phase 2: Product Backlog Refinement (Backlog Grooming)

Before a story can be pulled into a sprint, the development team, QA engineers, and the Product Owner hold a **Refinement Session**. During this meeting, they inspect the story against the **INVEST** criteria (ensuring it is independent, negotiable, valuable, estimable, small, and testable).

1. **Identifying Ambiguities (The Problem with Broad Stories):**
   - *Developer asks:* "What search criteria should be supported? Just title, or author and ISBN too? And how many results should show per page?"
2. **Refining and Splitting:**
   - The Product Owner clarifies the exact scope for the upcoming release.
   - They define the explicit **Acceptance Tests (Confirmation)** right on the back of the card (Section 3.3.1):
     - *Acceptance Test 1:* Search for books using an ISBN.
     - *Acceptance Test 2:* Search for books using an author's name.
     - *Acceptance Test 3:* Search for books using a title.
     - *Acceptance Test 4:* Search for books added on or after a specific date.
3. **Estimation:**
   - The team assigns a size/effort score (e.g.,*3 Story Points*) because the refined criteria make the work clear and small enough to complete within a sprint.

Phase 3: Sprint Planning & Moving to the Sprint Backlog

During the Sprint Planning meeting for **Sprint 1**, the team reviews the top of the Product Backlog.

- Because the book-search story has now been fully **refined** and estimated, the Product Owner pulls it from the master **Product Backlog** and places it into the **Sprint Backlog** (Sprint 1).

Phase 4: Execution on the Scrum Board (Task Breakdown)

Once inside the sprint, the development team takes the refined user story and breaks it down into technical implementation tasks on their **Scrum Board**:

- **Tasks Created for this Story:**
  1. *Database Task:* Add indexes for ISBN, title, and author fields in PostgreSQL.
  2. *Backend Task:* Implement REST API endpoint `GET /api/books/search?query=...` with filtering logic.
  3. *Frontend Task:* Design and code the search input bar and results grid component.
  4. *QA Task:* Write automated unit and integration tests covering Acceptance Tests 1–4.

As the team executes the work, the story flows across the Scrum board columns:

Phase 5: Review and Acceptance (Confirmation)

- Once all technical tasks are finished, the story moves to **Testing / Review**.
- The QA engineer or Product Owner runs through the pre-defined **Acceptance Tests (Confirmation)** specified during the refinement phase (e.g., typing an ISBN to ensure the correct book appears).
- Because all criteria pass successfully without "gold plating" (unauthorized extra features), the user story is officially marked as **Done** and is ready to be deployed in the working software increment at the end of the sprint!

7 Use Cases

In plan-driven or Waterfall software development, requirements engineering happens upfront in a dedicated, comprehensive phase before design and coding begin. Because projects following this methodology rely on detailed, stable documentation to mitigate costly rework, lightweight tools like agile user stories are often insufficient.

Use Cases bridge this gap, they serve as a formal, highly detailed textual specification of system behavior. They act as a clear, written agreement between stakeholders and the development team, ensuring everyone shares the exact same vision of what the system must achieve before engineering resources are committed.

> 📖 DEFINITION 3: **Use Case**
>
> A **use case** is a structured textual document that specifies the functional requirements of a system from an external **actor's** perspective.

> 📖 DEFINITION 4: **The Actor**
>
> An entity external to the system interacting with it to achieve a specific goal. Typically a human user, but it can also be another software system or external hardware component.

> ⓘ **Note**
>
> Use cases are written from the perspective of an **actor** interacting with the system to achieve specific objectives. Typically, the actor is a human user, although it can also be another software or hardware component. In all cases, the actor is an entity external to the system.

A use case breaks an operation down into two primary lists of steps:

1. **Main Flow (Happy Path):** The sequence of steps executed when everything goes successfully without errors or interruptions.
2. **Extensions:** Alternative paths, specialized variations, or error-handling steps triggered at specific points in the main flow.



WRITING EFFECTIVE USE CASES

Alistair Cockburn defines a use case as a contract for behavior that captures the interactions between a system and external actors to achieve a specific user goal. Rather than listing internal UI widgets or isolated technical specs, effective use cases focus on stakeholder value and behavioral requirements

OPEN PDF

7.1 The Formal Way to Write a Use Case

A well-structured, formal use case follows established industry guidelines (such as those by Alistair Cockburn) and typically incorporates the following standard format:

- **Title:** Must start with an active verb in the base form (e.g.,*Transfer Values between Accounts*).
- **Primary Actor:** The external entity initiating the use case.
- **Pre-conditions:** Conditions that *must* be true before the use case can start (e.g.,*Customer must be logged into the banking portal*).
- **Post-conditions:** State of the system after successful execution (e.g.,*Funds are debited from source and credited to destination account*).
- **Main Flow (Happy Path):** Numbered steps written in simple language, highlighting active actions taken by the actor and responses by the system. Included use cases are often underlined (e.g., `1 - Authenticate Customer`).
- **Extensions:** Alphabetical/numeric modifiers mapping back to specific main flow steps to handle alternative scenarios or errors (e.g., `3a - If transfer amount exceeds current balance, system requests new amount`).

> 📝 EXAMPLE 5: **Transfer Values between Accounts**
>
> **Transfer Values between Accounts**
>
> - **Actor:** Bank Customer
> - **Pre-conditions:** The customer is authenticated and has an active account balance.
> - **Main Flow:**
>   1. Authenticate Customer
>   2. Customer enters destination account and branch.
>   3. Customer enters the amount for transfer.
>   4. Customer specifies the transfer date.
>   5. System executes the transfer.
>   6. System asks if the customer wants to make another transfer.
> - **Extensions:**
>   - `2a` - If the account or branch is incorrect, system requests a new account and branch.

- `3a` - If transfer amount exceeds current balance, system requests a new amount.
- `5a` - If the transfer date is the current date, system processes the transfer immediately.

7.1.1 The core rules and good practices for writing effective use cases:

The core rules and good practices for writing effective use cases:

1. Structural and Syntax Rules

- **Name with a Verb:** Every use case name must start with a verb in the base form (e.g.,*Transfer Values between Accounts*).
- **Identify the Main Actor:** Clearly specify the external actor interacting with the system.
- **Include Other Use Cases:** You can include another use case within a flow. The syntax requires underlining the included use case's name (e.g., Authenticate Customer), meaning all its steps must execute before proceeding (similar to a macro).
- **Utilize Extensions Effectively:** Extensions serve two main purposes:
  1. Detailing a step in the main flow (e.g., branching behavior based on dates).
  2. Handling errors, exceptions, and cancellations (e.g., insufficient balance).
- **Avoid Decision Statements in Main Flows:** Avoid programming-like conditional statements (such as `if` or `repeat until`) in the main flow. Instead, branch normal alternatives and error checks out into **extensions**.

2. Recommended Sections

In addition to the main flow and extensions, use case descriptions can include:

- **Purpose:** What the use case achieves.
- **Pre-conditions:** Conditions that must be true *before* execution.
- **Post-conditions:** State of the system *after* successful execution.
- **Related Use Cases:** Links to dependent or included operations.

3. Good Practices for Writing Use Cases

- **Use Simple, Accessible Language:** Write as if explaining it to an early elementary school student. Each step should clearly state an active verb performed by the actor or the system (e.g.,*The customer inserts the card into the ATM*,*The system validates the inserted card*).

- **Keep It Concise:** Following Alistair Cockburn's recommendation (`[link](https://dl.acm.org/citation.cfm?id=517669)`), keep the **main flow to a maximum of nine steps**. If a use case gets too complex, break it down or group related steps together (e.g., combining *User enters login* and *User enters password* into *User enters login and password*).
- **Maintain a High Level of Abstraction:** Use cases are requirements documents,**not algorithms or pseudo-code**. Avoid procedural loops; instead, use descriptive natural language phrases like *"The customer browses the catalog until finding the desired product."*
- **Omit Technology and UI Details:** Do not mention specific user interfaces, button colors, hardware choices, or design aspects (e.g., avoid writing *"The customer presses the green button"*). Focus strictly on *what* the system should do, not *how* it will be implemented.
- **Avoid Trivial CRUD Use Cases:** Do not create separate, trivial use cases for every individual database operation like *Create Student*,*Retrieve Student*, etc. Instead, group them under a single comprehensive use case like *Manage Student*.
- **Enforce Consistent Vocabulary:** Use the exact same terms across all use cases (avoid mixing words like *Customer* and *User* interchangeably). It is recommended to maintain a project **glossary** as suggested in *The Pragmatic Programmer* to ensure common understanding between users and developers.

7.1.2 Examples

> 📝 **EXAMPLE 6: Use Cases for a Library Management System**
>
> Review the following high-level requirements for a Library Management System:
>
> - **FR-01:** The system must allow authenticated students and instructors to search for books by ISBN, author, title, or publication date.
> - **FR-02:** The system must allow students to borrow available books from the library.
> - **FR-03:** The system must allow students to return borrowed books.
> - **FR-04:** The system must allow students to renew active book loans.
> - **FR-05:** The system must allow students to reserve books that are currently out on loan.
> - **FR-06:** The system must allow instructors to request/recommend new books for library acquisition.
> - **FR-07:** The system must allow library staff members to register new users in the system.
> - **FR-08:** The system must allow library staff members to add new books or remove damaged books from the collection.

✅ Show Solution

Use Case 1: Search for Books

Use Case 1: Search for Books

- **Mapped Requirements:** FR-01
- **Primary Actor:** Library User (Student or Instructor)
- **Pre-conditions:** The user has access to the library catalog interface.
- **Main Flow:**
  1. User selects search criteria (ISBN, author, title, or date).
  2. User enters search keywords or parameters.
  3. System validates the search parameters.
  4. System queries the book catalog and displays matching results.
- **Extensions:**
  - `3a` - If search parameters are empty or invalid, system requests valid input.
  - `4a` - If no matching books are found, system displays a "No results found" message and suggests alternative keywords.

Use Case 2: Borrow Book

- **Mapped Requirements:** FR-02
- **Primary Actor:** Student
- **Pre-conditions:** Student is authenticated and has no active borrowing suspensions or overdue fines.
- **Main Flow:**
  1. Search for Books
  2. Student selects an available book from the search results.
  3. Student requests to borrow the selected book.
  4. System verifies that the book is available and the student's account is in good standing.
  5. System records the loan, updates the book status to on-loan, and generates a due date.
- **Extensions:**
  - `1a` - If the student is not logged in, system triggers Authenticate Customer.

- `4a` - If the book is currently unavailable, system informs the student and offers the option to reserve it via Reserve Book.
- `4b` - If the student has reached their maximum borrowing limit or has overdue fines, system rejects the loan and displays an error message.

Use Case 3: Reserve Book

Use Case 3: Reserve Book

- **Mapped Requirements:** FR-05
- **Primary Actor:** Student
- **Pre-conditions:** The desired book is currently out on loan to another user.
- **Main Flow:**
  1. Search for Books
  2. Student selects an unavailable book from the catalog.
  3. Student requests a reservation for the book.
  4. System checks student reservation eligibility.
  5. System adds the student to the book's reservation queue and confirms the reservation.
- **Extensions:**
  - `4a` - If the student has already reserved or currently holds a copy of the book, system rejects the duplicate reservation request.

Use Case 4: Manage Book Inventory

Use Case 4: Manage Book Inventory

- **Mapped Requirements:** FR-08
- **Primary Actor:** Library Staff Member
- **Pre-conditions:** Staff member is authenticated with administrative privileges.
- **Main Flow:**
  1. Staff member selects an inventory action (add new book or remove damaged book).
  2. Staff member inputs book details or scans book identification codes.
  3. System validates the provided book data.
  4. System updates the catalog database to reflect the inventory change.
- **Extensions:**

- `3a` - If mandatory book details are missing or formatted incorrectly, system requests correct information.
- `4a` - If a book marked for removal has active loans or pending reservations, system displays a warning and requires confirmation before archiving.

📝 EXAMPLE 7: **Multiple detailed use cases**

Functional Requirement

- **FR-01:** The system must allow users to manage their product orders in the online store.

(Notice how broad this requirement is—it covers placing, modifying, canceling, and tracking orders. Trying to force all of this into a single use case would violate Alistair Cockburn's rule of keeping the main flow concise and under 9 steps. Instead, we break it down into multiple distinct use cases.)

✅ Show Solution

Corresponding Use Cases (Multiple)

Use Case A: Place New Order

- **Primary Actor:** Online Store Customer
- **Pre-conditions:** Customer is authenticated, has items in the shopping cart, and has a saved delivery address.
- **Main Flow:**
  1. Customer views the items in the shopping cart and initiates checkout.
  2. Customer confirms the delivery address and shipping method.
  3. Customer selects the payment method.
  4. Process Payment
  5. System generates an order confirmation number and sends a confirmation receipt.
- **Extensions:**
  - `4a` - If payment authorization fails, system prompts the customer for a different payment method.

Use Case B: Cancel Pending Order

- **Primary Actor:** Online Store Customer

- **Pre-conditions:** Customer is authenticated and has an active order that has not yet entered the shipping phase.
- **Main Flow:**
  1. Customer views their order history.
  2. Customer selects the specific pending order to cancel.
  3. Customer requests to cancel the order and provides a cancellation reason.
  4. System verifies that the order status permits cancellation.
  5. System cancels the order, restores inventory levels, and initiates a refund if applicable.
- **Extensions:**
  - `4a` - If the order has already been shipped or processed for delivery, system rejects the cancellation request and displays an explanatory message.

Use Case C: Track Order Status

- **Primary Actor:** Online Store Customer
- **Pre-conditions:** Customer has at least one placed order.
- **Main Flow:**
  1. Customer opens their order tracking dashboard.
  2. Customer selects an order from the list.
  3. System retrieves the latest shipping and fulfillment status from the logistics partner.
  4. System displays the current tracking timeline and estimated delivery date to the customer.
- **Extensions:**
  - `3a` - If the tracking service connection fails, system displays the last known cached status and a "Data temporarily unavailable" warning.

EXAMPLE 8: Multiple use cases derived from a single functional requirement

You are specifying requirements for an online banking platform using plan-driven development. Consider the following high-level functional requirement:

- **FR-01:***The system must allow authenticated customers to manage their bank accounts, including viewing account statements and transferring funds to other accounts.*

Questions:

1. Explain why a broad functional requirement like **FR-01** should be broken down into **more than one use case.**

2. Write **two distinct formal use cases** to fulfill **FR-01**:

- **Use Case A:** *View Account Statements*
- **Use Case B:** *Transfer Values between Accounts*

Ensure both use cases adhere to all structural rules:

- Use verb-first base names.
- Keep the main flow under 9 steps.
- Avoid procedural conditional statements (`if` / `else`) in the main flows, placing error handling and alternatives into the **extensions** section.
- Properly include the security requirement by utilizing the `<u>Authenticate Customer</u>` included use case with correct underlining syntax in Step 1 of both flows.

✅ Show Solution

1. Why FR-01 Expands into Multiple Use Cases

Attempting to cram viewing statements, fund transfers, and other account management tasks into a single use case would result in an overly complex, lengthy document that violates this principle and obscures specific user goals. Therefore, broad functional requirements must be decomposed into distinct, focused operational units.

2. Formal Use Cases

Use Case A: View Account Statements

- **Primary Actor:** Bank Customer
- **Pre-conditions:** The system is operational and connected to the account ledger database.
- **Main Flow:**
  1. Authenticate Customer
  2. Customer selects the account statement option from the main menu.
  3. Customer specifies the desired billing month and year.
  4. System queries the database and retrieves the statement records.
  5. System displays the formatted account statement to the customer.
- **Extensions:**
  - `1a` - If authentication fails or credentials are invalid, system displays an error message and terminates the use case.
  - `1b` - If the customer has not logged in during the active session, system triggers Authenticate Customer automatically.

- `4a` - If no statement records exist for the selected period, system displays a "No records found" notification.

Use Case B: Transfer Values between Accounts

- **Primary Actor:** Bank Customer
- **Pre-conditions:** The customer is authenticated and has an active account balance.
- **Main Flow:**
  1. Authenticate Customer
  2. Customer enters destination account and branch.
  3. Customer enters the amount for transfer.
  4. Customer specifies the transfer date.
  5. System executes the transfer.
  6. System asks if the customer wants to make another transfer.
- **Extensions:**
  - `1a` - If authentication fails, system terminates the use case.
  - `2a` - If the account or branch is incorrect, system requests a new account and branch.
  - `3a` - If transfer amount exceeds current balance, system requests a new amount.
  - `4a` - Transfer date must be the current date or within one year in the future.
  - `5a` - If the transfer date is the current date, system processes the transfer immediately.
  - `5b` - If the transfer date is in the future, system schedules the transfer.

8 Use Case Diagrams

While the core substance of requirements modeling lies in textual specifications, the Unified Modeling Language (UML) introduces the **Use Case Diagram** to provide a high-level visual perspective of a system.

📖 DEFINITION 5: **Use Case Diagram**

A **Use Case Diagram** serves as a graphical "visual catalog" or architectural blueprint of a system. It maps out:

- **System Boundary:** Represented as a rectangle enclosing the system features, separating internal software operations from external entities.

- **Actors:** Represented as stick figures positioned *outside* the system boundary, illustrating the human users, hardware, or external systems that interact with the software (e.g.,*Customer*,*Manager*).
- **Use Cases:** Represented as horizontal ellipses inside the boundary box, depicting the primary functional goals or features the system provides.
- **Relationships:** Connecting lines that show participation between actors and use cases, or specialized structural relationships such as `<<includes>>` (as seen in the banking system where *Transfer Funds* includes *Authenticate Customer*).

8.1 Use Case Diagrams vs. Textual Use Cases

- **Use Case Diagrams (The Visual Map):** You typically maintain **one master diagram** (or a few high-level architectural ones) for the entire system. It provides an executive overview of who interacts with the system and what high-level features exist, but it contains **zero step-by-step logic**.
- **Textual Use Cases (The Detailed Chapters):** You write a separate, rigorous text document for *each distinct user goal or feature* (e.g.,*View Account Statements*,*Transfer Values between Accounts*). If a system has 20 functional goals, you maintain 20 distinct textual documents.

> ⓘ **Use Cases vs. Use Case Diagrams**
>
> - **Textual Use Cases (1-to-1 Mapping):** If your system has 20 distinct functional requirements or user goals, you will write **20 separate textual use case documents**, each detailing its own pre-conditions, main success flows, and extension paths.
> - **Use Case Diagrams (Many-to-1 Mapping):** You typically maintain **only one master visual diagram** (or a few high-level architectural ones) enclosed within a system boundary box. This single UML diagram acts as a catalog, mapping out all actors and use cases together in a single bird's-eye view.

8.1.1 When to Use Each Approach

In a rigorous software engineering process, textual documents and visual diagrams complement each other rather than compete. You use them at different stages and for different purposes:

- **When to Use Use Case Diagrams:**
  - **At Project Inception:** To scope the project, identify key system boundaries, and map out major stakeholder interactions during early planning.

    - **For Executive Overviews:** When presenting the system architecture to high-level stakeholders or project sponsors who need a bird's-eye view of functional modules without getting bogged down in procedural steps.

- **When to Use Textual Use Cases:**
    - **During Detailed Requirements Specification:** Whenever developers and customers need to reach a precise, binding agreement on exactly how a feature must behave
    - **During Development and Testing:** When engineers need to implement exact operational workflows, handle error states via extensions (`2a` , `3a`), and write acceptance test cases.

8.2 Use Case Diagram objects

At a high level, a Use Case Diagram consists of four fundamental building blocks, as outlined by standard Unified Modeling Language (UML) specifications:

1. **Actor**: Actor in a use case diagram is any entity that performs a role in one given system. This could be a person, organization or an external system and usually drawn. Actors are strictly external to the system boundary and can be categorized as *primary* (users who achieve a direct goal),*supporting* (systems/services that provide assistance), or *offstage* (stakeholders interested in outcomes but not directly executing actions).
2. **Use Case:** A specific function, service, or task that the system performs to yield a measurable value or goal for an actor. Drawn as a horizontal ellipse and typically named using a verb-noun phrase (e.g.,*Check Account Balance*,*Place Order*).
3. **System Boundary:** A rectangular boundary box that delineates the scope of the software system being developed.It clearly separates what falls inside the automated system capabilities from what remains outside in the external environment (where actors reside).
4. **Package:** An optional grouping mechanism used in complex, enterprise-scale projects to organize related use cases into manageable subsystems or modules.

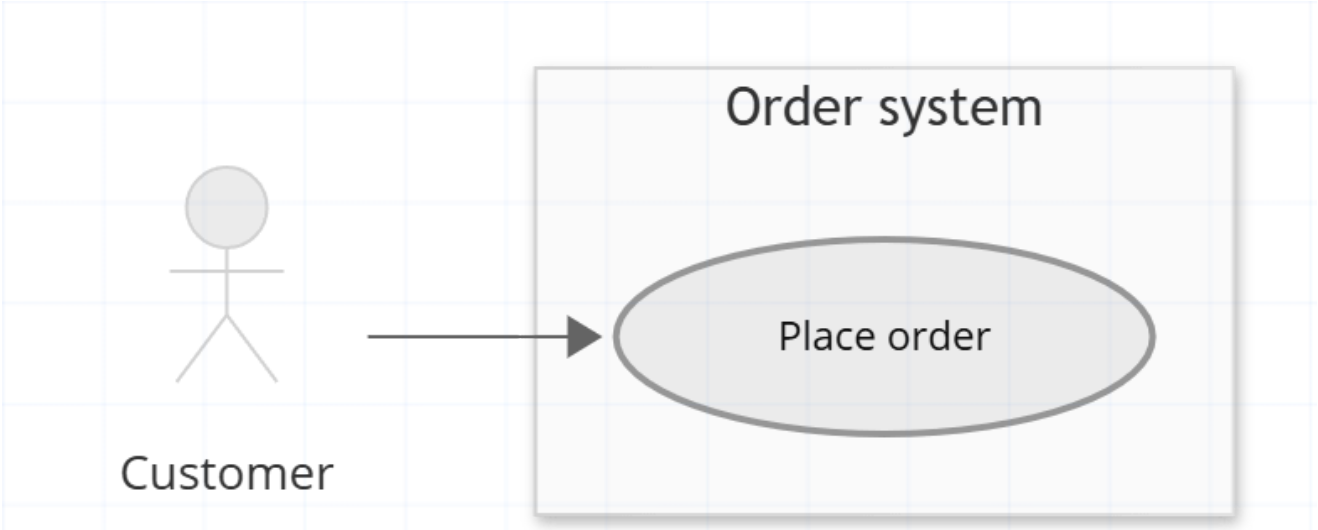


Diagram 1: Use Case Diagram Components

EXAMPLE 9:

A web application allows a user to:

- read the products in the catalog
- search products in the catalog
- buy a product

Make a use case diagram to model this system.

Show Solution

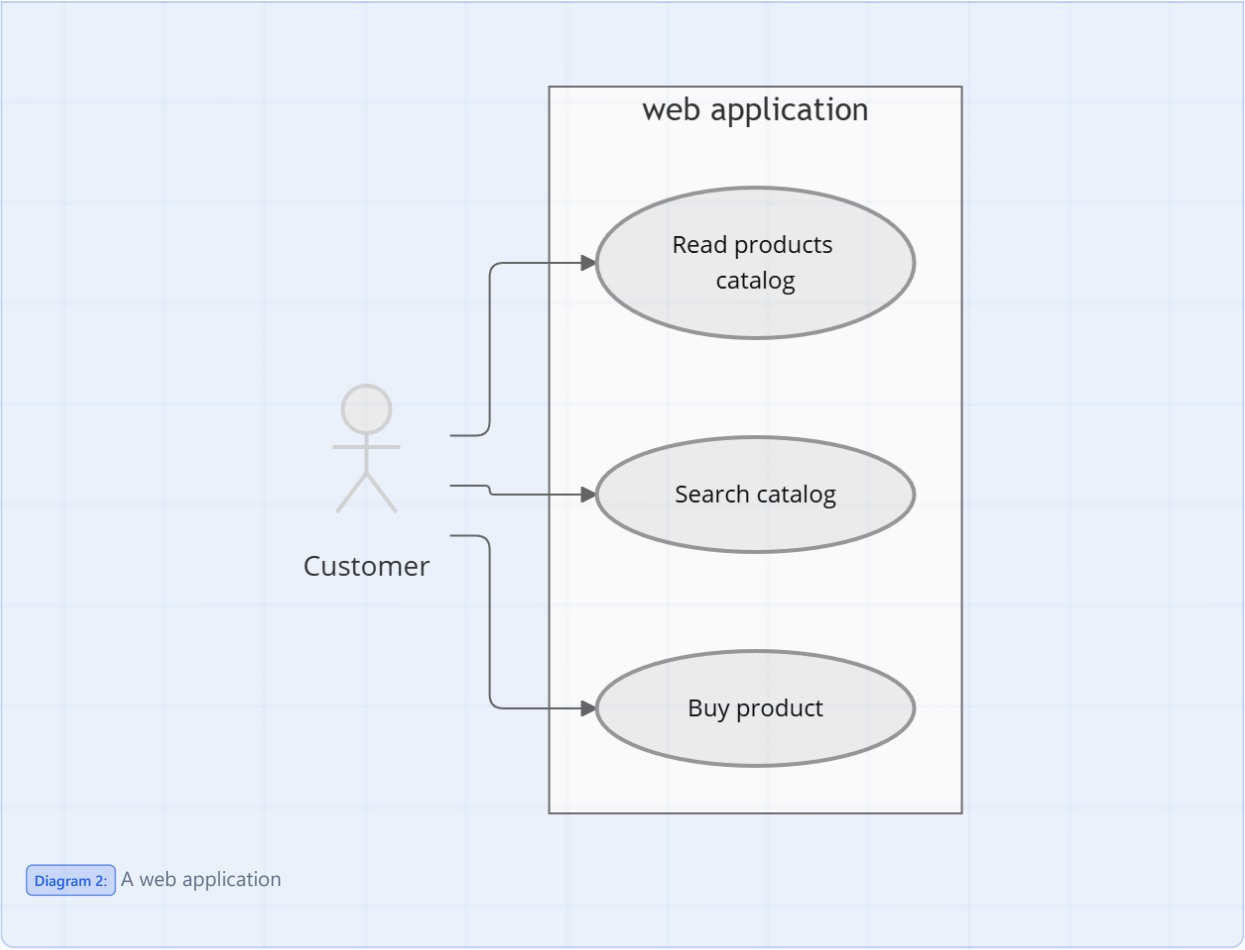


Diagram 2: A web application

8.2.1 Use Case Diagram Relationships

In a Use Case Diagram, relationships play a crucial role in depicting the interactions between actors and use cases.

8.2.1.1 Association Relationship

Association Relationship shows the interaction between an actor and a use case. It is represented by a line connecting the actor and the use case.

EXAMPLE 10:

- **Actor:** Customer
- **Use Case:** Transfer Funds
- **Association:** A line connecting the "Customer" actor to the "Transfer Funds" use case, indicating the customer's involvement in the funds transfer process.

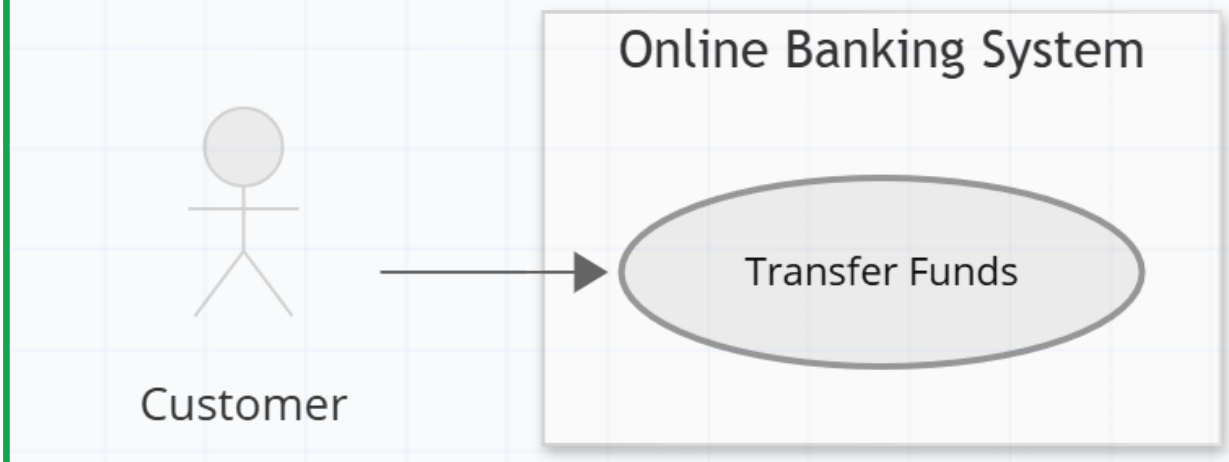


Diagram 3: Transfer Funds

8.2.1.2 Include Relationship

An **include** relationship (sometimes called *sub-use case* or *uses*) specifies that the behavior of a **base use case** explicitly incorporates all the behavior defined in an **included use case**.

- **When to use:** Use it to represent mandatory, shared behavior that occurs in multiple use cases, allowing you to factor out common steps and prevent duplication.
- **Execution:** The execution of the included use case is mandatory whenever the base use case runs. All steps of the included use case must be completed before the system proceeds.
- **Notation (UML):** Represented by a dashed arrow pointing from the base use case to the included use case, labeled with the keyword `<<include>>`.

EXAMPLE 11:

A high-level overview of the banking system's primary workflows. It shows that Customers can withdraw money and transfer funds, while Managers can open accounts. Both customer-facing financial transactions require mandatory customer authentication (`<<include>>`) to ensure system security.

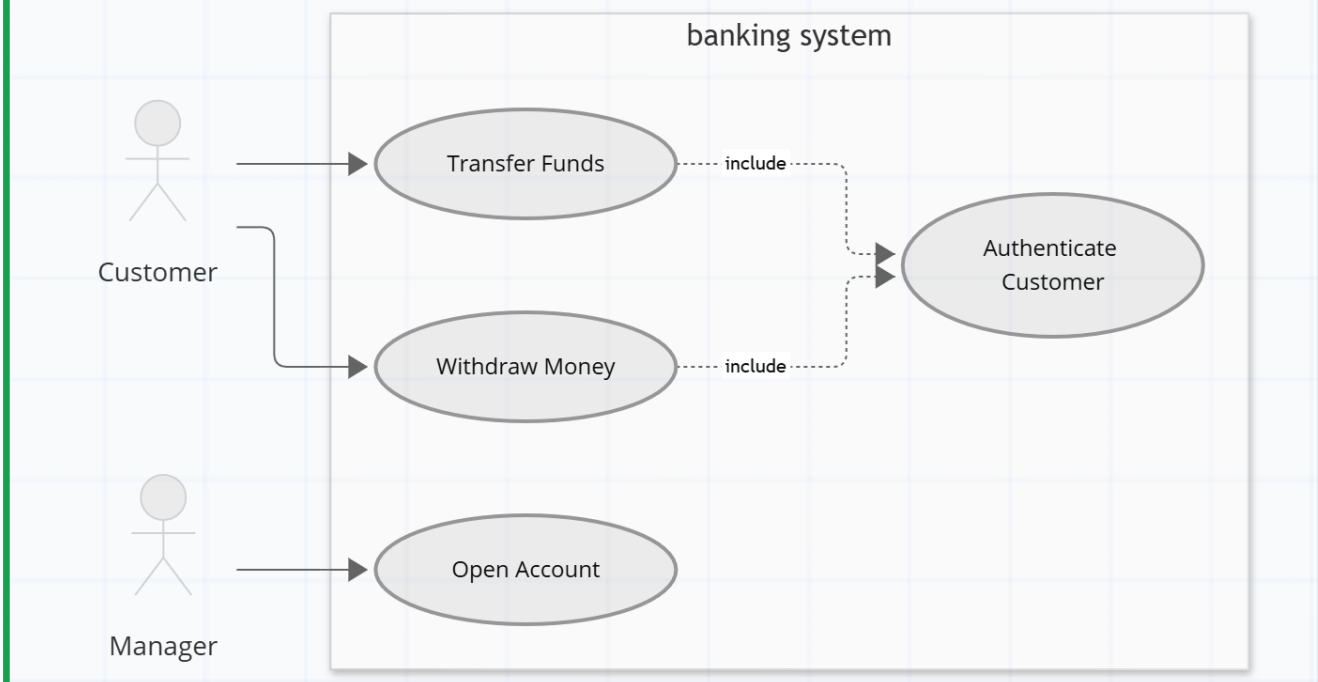


Diagram 4: Core Banking System Use Cases

8.2.1.3 The Extend Relationship

An **extend** relationship specifies that an **extension use case** may optionally augment or insert behavior into a **base use case** under certain conditions (such as handling errors, optional features, or specific alternate paths).

- **When to use:** Use it to model optional behavior, special-case scenarios, or exception handling that *does not* always happen, keeping the main "happy path" clean and uncluttered.
- **Execution:** The base use case can execute completely on its own without the extension ever running. The extension only triggers if a specific condition (an "extension point") is met.
- **Notation (UML):** Represented by a dashed arrow pointing *from* the extension use case *back to* the base use case, labeled with the keyword `<<extend>>`.

EXAMPLE 12:

This use case diagram models the checkout workflow of an e-commerce platform. The primary actor is the **Customer**, who initiates the base use case **"Checkout Order"**.

To ensure a successful transaction, the system mandatorily executes the **"Process Payment"** use case, modeled with an `<<include>>` relationship (as payment is a required step).

However, the diagram also features two `<<extend>>` relationships: **"Apply Promo Code"** and **"Request Gift Wrapping"**. Unlike `<<include>>`, an `<<extend>>` relationship represents *optional* behavior. These extended use cases only execute under specific conditions (e.g., if the

customer actively chooses to enter a discount code or selects gift wrapping). They add functionality to the base "Checkout Order" use case without being mandatory for every transaction.

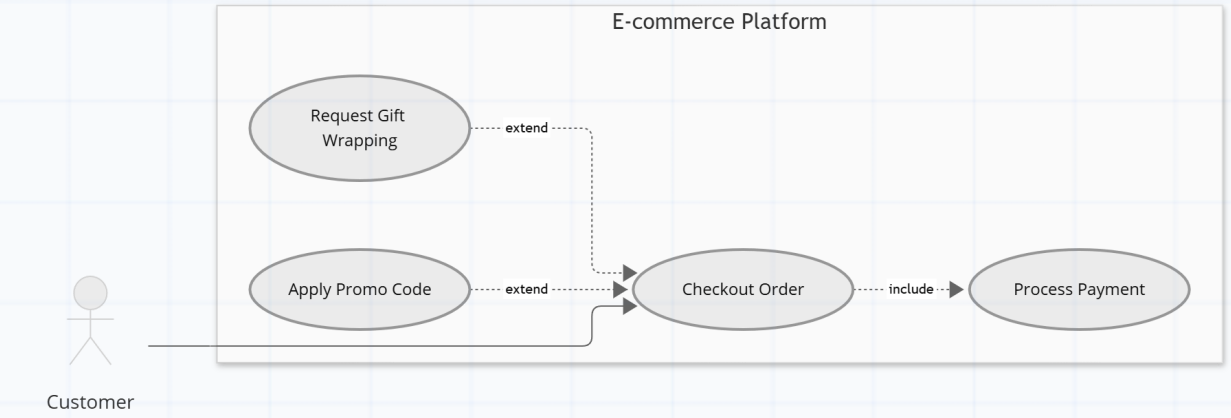


Diagram 5: E-Commerce Checkout Process Use Case Diagram

extend vs include

- `<<include>>` **(Mandatory)**: The base use case *cannot* complete without it. (e.g., You cannot checkout without processing payment). Syntax: `Base ..> : include RequiredStep`
- `<<extend>>` **(Optional)**: The extending use case adds extra behavior to the base use case *only if* certain conditions are met. (e.g., You can checkout perfectly fine without a promo code, but if you have one, this step extends the flow). Syntax: `OptionalStep ..> : extend Base`

8.2.1.4 Solved Examples

EXAMPLE 13: Online Quiz Management System — Use Case Model

A university is developing an Online Quiz Management System to facilitate automated testing and quality-controlled question authoring. The system involves three distinct types of users: **Students** (who take quizzes), **Tutors** (who author content), and **Examinators** (who validate content).

When a Student requests a quiz, the system automatically retrieves a set of questions from its database, composes them into a cohesive quiz, and presents them. After the Student submits their answers, the system automatically grades them. During the quiz, if a Student is stuck, they may optionally request a hint.

Functional Requirements

- **FR-1:** The system shall allow a Student to request and initiate a quiz session.

- **FR-2:** The system shall automatically select a predefined number of questions from the database to compose the quiz.
- **FR-3:** The system shall automatically evaluate and rate the Student's answers upon submission.
- **FR-4:** The system shall allow a Student to request a hint for the current question during an active quiz.
- **FR-5:** The system shall allow a Tutor to create and submit new questions to the question bank.
- **FR-6:** The system shall allow a Tutor to create and associate hints with existing questions.
- **FR-7:** The system shall allow an Examinator to review pending questions and mark them as "Certified" or "Rejected".

Business Rules

- **BR-1:** The system shall only select questions that have a "Certified" status when composing a quiz.
- **BR-2:** A question cannot be marked as "Certified" until an Examinator verifies it is not trivial and is logically sensical.
- **BR-3:** Hints are strictly optional; the quiz flow must not be interrupted or delayed if a Student does not request a hint.

Non-Functional Requirements

- **NFR-1 (Security):** The system shall enforce Role-Based Access Control (RBAC) to ensure Students, Tutors, and Examinators can only access their respective functionalities.
- **NFR-2 (Performance):** The system shall compose and load a 50-question quiz within 2 seconds of the Student's request.

To maintain quality, Tutors are responsible for writing new questions and hints. However, these cannot be used immediately. An Examinator must review and "certify" each question to ensure it is logically sound, unambiguous, and of appropriate difficulty. Only certified questions are eligible to be picked for a Student's quiz.

✅ Show Solution

Table 3: *Actor Definitions*

Actor	Description
User	A student or learner who takes quizzes to test their knowledge
Tutor	An educator who creates and provides questions and hints for the quiz database
Examinator	A quality assurance role responsible for reviewing and certifying questions before they can be used in quizzes

Table 4: *Use Case Relationships*

Relationship Type	Source Use Case	Target Use Case	Reason
Include	Take Quiz	Pick Questions from DB	Mandatory: Quiz cannot exist without questions
Include	Take Quiz	Compose Quiz	Mandatory: Questions must be assembled into a quiz format
Include	Take Quiz	Rate Answers	Mandatory: All submissions must be graded
Extend	Request Hint	Take Quiz	Optional: Hints only shown if user requests them

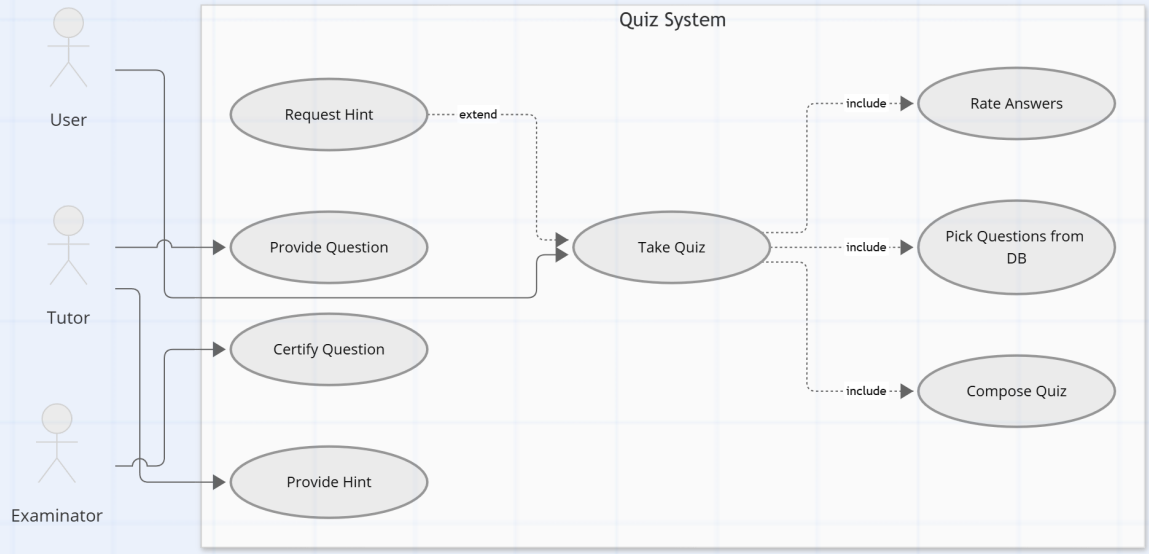


Diagram 6: Online Quiz Management System — Use Case Model

gdfgfd

Untitled Section